



SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

An Overlay for Social Networks based on Directed Acyclic Graphs on the Edge

Dan Bachar





SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

An Overlay for Social Networks based on Directed Acyclic Graphs on the Edge

**Ein Overlay für soziale Netzwerke auf der
Grundlage gerichteter azyklischer Graphen auf
der Kante**

Author:	Dan Bachar
Examiner:	Prof. Dr.-Ing. Jörg Ott
Supervisor:	Prof. Dr. Idit Keidar
Advisor:	Dr. David Guzman
Submission Date:	15.8.2026



I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 15.8.2026

Dan Bachar

Acknowledgments

Abstract

Recent crises, natural disasters, and large-scale protests have highlighted the significant reliance of people on online social networks and instant messaging platforms for communication. During crises, consolidated communication systems can become strained, partially or wholly destroyed, censored, or otherwise monitored. Permissionless distributed overlays are alternatives to consolidated communication realizations. System designers consider these overlays to be more resilient, since they distribute power and content control among many equal peers. These overlays introduce a massive performance overhead, as they rely on gossip and constant chatter between peers to disseminate the correct system state by reaching consensus, through many rounds of communication. This performance overhead poses a risk to usability of such systems in disaster zones, as peer devices often have limited connectivity and power supply, which together with the gossip protocol, can severely limit the peers' ability to communicate. In this paper, we propose a system architecture for unstructured peer-to-peer networks that achieves grassroots constitutional consensus based on efficient geometrical overlay design that trade achieve a higher throughput by using more edge computing. We use this system architecture to evaluate a decentralized social network deployed on smartphones using BLE connectivity, where users have total control of their data and privacy.

Kurzfassung

Recent crises, natural disasters, and large-scale protests have highlighted the significant reliance of people on online social networks and instant messaging platforms for communication. During crises, consolidated communication systems can become strained, partially or wholly destroyed, censored, or otherwise monitored. Permissionless distributed overlays are alternatives to consolidated communication realizations. System designers consider these overlays to be more resilient, since they distribute power and content control among many equal peers. These overlays introduce a massive performance overhead, as they rely on gossip and constant chatter between peers to disseminate the correct system state by reaching consensus, through many rounds of communication. This performance overhead poses a risk to usability of such systems in disaster zones, as peer devices often have limited connectivity and power supply, which together with the gossip protocol, can severely limit the peers' ability to communicate. In this paper, we propose a system architecture for unstructured peer-to-peer networks that achieves grassroots constitutional consensus based on efficient geometrical overlay design that trade achieve a higher throughput by using more edge computing. We use this system architecture to evaluate a decentralized social network deployed on smartphones using BLE connectivity, where users have total control of their data and privacy.

Contents

Acknowledgments	iii
Abstract	iv
Kurzfassung	v
1. Introduction	1
2. State-of-the-Art	5
2.1. Proximate Peer-to-peer Communication Technologies	5
2.1.1. Wi-Fi	6
2.1.2. Alternatives	7
2.1.3. Bluetooth	7
2.2. Remote Peer-to-peer Communication Technologies	8
2.2.1. Application Protocols: HTTP, HTTPS, QUIC	8
2.2.2. Transport-level Protocols: TCP, UDP	10
2.2.3. UDX	10
2.3. Opportunistic Networking Systems	11
2.3.1. Background	11
2.3.2. Routing in MANETs and OppNets	12
2.4. Decentralized Social Networks	14
2.5. Grassroots Social Networks	17
3. System Design	19
3.1. Environment Assumptions	19
3.2. Design Goals	20
3.3. Architecture	21
3.3.1. Identity	22
3.3.2. Announce	22
3.3.3. BLE Advertising & Scanning	23
3.3.4. Trust levels: Friend or Foe?	23
3.3.5. Cold-calls, Remote Connection Facilitation: Rendezvous	24
3.3.6. Encryption, Key Exchange, Noise Security	24
3.4. System Architecture	25

4. Testbed	26
4.1. Wireless Opportunistic Networks as a Bridge to Collective Communication . .	26
4.1.1. Fundamentals of Collective Communication	26
4.2. Control plane: RSSI and link establishment on a line	27
4.3. Data plane: throughput of a dilating clique	28
5. Evaluation	29
6. Discussion	30
7. Conclusion	31
A. Appendix	32
B. GLP Networking API Specification	33
B.1. Introduction	33
B.2. Networking API	33
B.2.1. Identity	33
B.2.2. Peer Discovery and Connection	34
B.2.3. Cold Calls and Trusted Environment Levels	35
B.2.4. Point-to-Point Communication	35
B.2.5. Networking Assumptions	35
B.3. madGLP Communication Model	36
B.3.1. Global Links and Messages	36
B.3.2. Message Types	36
B.3.3. Message Flow	36
B.4. madGLP-to-Networking Mapping	37
B.4.1. Agent Identity Mapping	37
B.4.2. Message Serialization	37
B.4.3. The Send Operation	38
B.4.4. The Receive Operation	39
B.5. BLE Medium	39
B.5.1. BLE Identity and Advertising	39
B.5.2. BLE Discovery	39
B.5.3. BLE Communication	39
B.6. IP Medium	40
B.6.1. IP Connection	40
B.6.2. IP Cold-Call	40
B.6.3. IP Communication	41
B.7. Rendezvous	41
B.7.1. Rendezvous Agents	41
B.7.2. Friends-Based Rendezvous	42
B.7.3. Address Maintenance	43
B.7.4. Rendezvous Protocol	43

B.7.5. System Predicates	44
B.7.6. GLP Implementation	45
B.7.7. Failure and Fallback	46
B.8. Public Identity and Access	47
B.8.1. Identity Record	47
B.8.2. Access Record	47
B.8.3. Publication Choices	48
B.9. Complete API Summary	48
B.9.1. Correspondence to IsolateManager	49
.1. Rendezvous Agent — GLP Implementation	49
.2. Rendezvous Client — GLP Implementation	51
.3. Friends-Based Rendezvous — GLP Implementation	52
.4. IP Cold-Call Invite Redemption	54
.4.1. Rendezvous-Gated Redemption	54
List of Figures	56
List of Tables	57
Bibliography	58

1. Introduction

Autocratic regimes have been employing internet shutdowns as an efficient means to suppress democratic protests, leading to fragmentation in the coordination of civil actions and state-sponsored violence [1, 2, 3]. In addition to physical repression, a "networked autocracy" [4], where regimes integrate controls over the media, finance, and information, to oppress legitimate opposition to them without costs to their reputation. Recent crises highlight this trend: during widespread protests in Iran early 2026 in Iran, triggered by the economic collapse of the currency (Rial) leading vendors in the main markets of Tehran to protest, the Islamic Republic has severed international prospects of the ongoing by suppressing access to the Internet, which imposed near-total blackout to create an information vacuum [1]. Similar practices have been utilized to stifle democratic movements globally, including during the 2025 protests in Nepal against a government ban on major social media platforms, the 2026 elections in Uganda, the 2024 student protests in India, and more [4, 5, 6]. In addition to policial repression engaged by the government, physical distrupctions can also limit free speech. As evident from communication disruptions due to airstrikes on main fiber optic lines, and the depletion of fuel for generators, information flow was severely limited to the population of Gaza since the start of the Israel-Hamas war in October 2023, halting humanitarian coordination, and leaving the affected population entirely cut off from life-saving information and assistance [7, 8]. Together, the aforementioned practices have led to a growing interest in peer-to-peer (p2p) messaging platforms, decentralized social networks [9, 10], and distributed ledger-based applications [11, 12].

Mobile Ad-Hoc Networks (MANETs) are infrastructure-less networks established using an array of battery-powered mobile devices that use the ad-hoc mode of their wireless network adapter to establish a spontaneous end-to-end connected, packet-switched network [13, 14]. In these networks, nodes act both as routers and hosts for data packets. MANETs, as well as Vehicle Ad-Hoc Networks (VANETs) and Smartphone Ad-Hoc Networks (SPANs), have been extensively studied, with research going back to DARPA's usage of radio to transmit data packets in the Packet Radio Network (PRNET) [15]. These elements were bulky, expensive, provided poor data rates, and handled node mobility poorly. Since their onset, MANETs have advanced to VANETs, which deal with rapid topology changes arising due to a network established between fast-moving vehicles such as drone nets [13], and SPANs, which focus on establishing networks around humans utilizing a commonly available resource, the smartphone. SPANs utilize commonly available infrastructure in normal commercial-grade smartphones to connect users without external infrastructure, by leveraging technologies like Wi-Fi Aware, Wi-Fi Direct [16], Bluetooth Classic, and Bluetooth Low Energy [17], to facilitate critical communication during natural disasters, wars, protests, or Internet outages. Because they rely on no external infrastructure, SPANs are oftentimes proposed as the foundation

to serverless grassroots social networks such as Bitchat [18], and indeed the idea behind SPANs and is tangential to the core of this research work: MANETs today offer a generally higher data rate than DTNs, but their requirement of a continuously-connected end-to-end path between source and destination make it impossible here, as oftentimes users suffer from offline periods due to emergencies. To route packets, MANETs usually relay on protocols that use topological knowledge of the nodes to efficiently route between them, being reactive to changes or working proactively to evaluate link states, and use out-of-band information like geological location [19]. An alternative to MANETs, DTNs have been proposed in communication-challenged regions where end-to-end path connectivity is lacking: research on DTNs was propelled by NASA for interplanetary travel; the Curiosity rover on Mars, a resource-constrained robot that has been travelling the surface of the red planet since 2012, has been using a DTN to convey pictures to Earth through a satellite orbiting the planet. Instead of wasting precious power on beaming a transmission directly to Earth, the Curiosity rover stores the transmission until the satellite is optimally located orthogonally over it, before sending the satellite its data packets. The satellite then carries the packets until it is at an optimal orbit to Earth. The final transmission between the satellite and Earth completes the communication cycle, enabling the rover to continue to explore Mars for ten years after the end of its intended two-year mission. The concept that thus lies at the core of DTNs is the Store-Carry-Forward paradigm, which stands in stark contrast to the immediacy requirements of normal packet routing: in standard packet routing, all nodes participating in a transmission between source and destination need to be connected and online concurrently; in DTN routing, the networking graph can be partially offline for extensive periods of time while nodes carry the transmissions. Similarly to MANETs, DTNs also suffer from communication overheads that arise due to control messages, and are especially exacerbated by dense environments with many participants. In addition to space travel, DTNs have also enabled nomadic populace in remote, hard-to-access environments lacking infrastructure the access to communication. The Laponia region near the Arctic Circle, homeland of the Sámi people, is characterized as a sparsely-populated region full of icy fjords, deep valleys, glaciers and mountains. The inhabitants of this region have traditionally been reindeer-herders, and often migrate with the reindeer herds along the yearly cycle. Due to the harsh terrain, sparse level of population, and partial autonomy of the Sámi council, the Sámi people lack reliable cellular and internet infrastructure. To provide the Sámi people with modern communication options, the traditional packet-switching paradigm of communication as conceptualized by the Internet is not optimal. Instead, Lindgren et al. propose the the Store-Carry-Forward paradigm using DTNs [20], where hosts store messages until an opportunity arises to forward them, carry them while moving until they reach a rendezvous with another node or a network attachment point, and then forward them upon contact. This paradigm enables the opportunistic usage of technologies such as Bluetooth to forward messages, which is ubiquitous to virtually all Smartphones and computers today. Insofar as the Internet is not available, DTNs using Store-Carry-Forward networking can be used as a fallback, being supplemented by traditional Internet infrastructure as it becomes available.

To guarantee free speech in disaster zones, during protests, and for citizens living under oppressive regimes, we decide to focus on unstructured p2p networks, as they prove to be more efficient in high-mobility scenarios characterized by the movement patterns of people, where the network topology is characterized by a high churn rate and a high rate of change, in comparison to structured p2p overlays. Structured p2p networks usually require an expensive randomized approach to establish resilient topologies, as they need to spread information across random peers to prevent dependence on any single peer [21, 22, 23]. Additionally, iterative diffusion of the information causes data plane overhead [22], which increases the message propagation latency [21, 24]. Popular blockchains, which use a structured p2p overlay, employ concentration of services in a few highly available nodes, poses a threat to resilience despite a high reliability of those nodes, as a single network partition can prevent the entire network from communicating [25].

Unstructured p2p networks, on the other hand, are more suitable for highly-dynamic alternative communication methods, often necessary to guarantee free decentralized communication means, as they usually do not distribute central information amongst the participating nodes. Most notably, the rise of unstructured p2p messaging platforms like Firechat [26], Briar [27], bittchat [18], and Amigo [28], have proven that there is a growing shift in user interest patterns away from traditional Centralized Social Networks towards diverse p2p social networks.

After exploring the different types of p2p overlay networks, we discuss the choice of the communication protocol, which dictates the limited physical device communication radius. We chose to focus on a stable protocol that most modern devices support and the populace is most likely to use in disaster zones: Bluetooth, and more specifically Bluetooth Low Energy (BLE) [17]. We decided to focus on Bluetooth Low Energy (BLE) as it is more power efficient, over classic Basic Rate/Enhanced Data Rate (BR/EDR), as BLE conserves device battery life, it allows users to communicate over short ranges over a longer period of time than comparable communication protocols e.g. WiFi Direct [29].

Next, we discuss the bottleneck of unstructured p2p networks: the communication algorithm. Most unstructured p2p networks rely on gossip protocols to disseminate information across the network, which reduces its performance and poses a threat to resilience (cite guided research paper?). Gossip protocols usually require constant chatter between peers to reach consensus about the system state, which introduces a massive performance overhead. This performance overhead poses a risk to usability of such systems in disaster zones, as peer devices often have limited connectivity and power supply, which together with the gossip protocol, can severely limit the peers' ability to communicate. The introduction of permissioned peers, such as high-availability bootstrapping nodes or signaling servers, improves the network's overall ability to communicate, bootstrap new communication links as well as maintain existing ones. However, those nodes pose an inherent tradeoff between performance and resilience to failure or surveillance, as relying on such nodes in disaster zones poses a risk with the infrastructure being oftentimes affected by network partitions, political censorship, or physical disruptions which halts the entire network from communicating. While such bootstrapping nodes ease the process of joining the network, discovering peers, and

(potentially) disseminating and ordering state, they weaken the peers' ability to function without their presence, which is, as we show in this paper, oftentimes unnecessary.

In this research paper, we deal mainly with the following three axes of distributed systems: security, decentralization, and resilience. In the first axis, security, we discuss the security of an open distributed system, its permission model, anonymity, confidentiality, and encryption. Following the security axis, we explore decentralization as a way to circumvent centralized control and avoid single points of failure. Several design decisions in our system architecture influence the system's decentralization: a pure p2p system completely resides on the peer devices; while this approach highlights communication which is independent from any existing networking architecture, it introduces limitations on the range of communication, as it only allows for communication based on proximity due to the physical device constraints. Finally, we discuss resilience, which in our context refers to network survivability under duress. These communication issues include Byzantine behavior, which is independent of the communication. These issues include reachability, bandwidth limitations, and coverage loss.

This paper proposes a system architecture for resilient unstructured p2p networks, based on a wireless dual communication stack that leverages both a short-range city-scale relation as well as as Internet-scale relation, when available, to facilitate effective communication. For this, in chapter 2, we present the p2p communication protocols we used: BLE 2.1 and UDX 2.2, as well as related works which lay at the foundation of this research paper: Peer-to-Peer Opportunistic Networking 2.3, Routing in Opportunistic Networking 2.3.2, Decentralized Social Networks (DSNs) 2.4, and finally, Grassroots Social Networks, and what sets them apart from DSNs 2.5. Finally, the State-of-the-Art chapter positions this research work contrastingly. To formalize the setting, goals, and environment for this research paper, chapter 3 will introduce our basic environment assumptions, an analysis of the concrete problem at hand, the design goals of the system, and finally introduce the system architecture 3.3 designed to support a permissionless, unstructured p2p social network, emphasizing the base algorithms, system components, and different communication layers. The 4 section introduces the testbed we used to evaluate the performance of the system architecture introduced in section 3.3. Next, chapter 5 evaluates the results of the experiment conducted on the testbed, which proves the possibility of our design. Chapter 6 discusses the implications of the results introduced in the previous chapter, and finally, chapter 7 concludes the paper.

2. State-of-the-Art

The State-of-the-Art section presents the papers that lay at the foundation at this research work, and positions it adverserally. The sections in this chapter range will provide an overview over the communication mediums that we considered for this work: Bluetooth Classic, Bluetooth Low Energy, Bluetooth Mesh, Wi-Fi ad-hoc and Direct, and others. Next, this section continues with elaborating the networking assumptions that make this work possible, primarily discussing the difference between traditional packet-switching and the Store-Carry-Forward paradigm, that enables Delay-Tolerant Networking, while also presenting similar systems that have been developed using this paradigm. Followingly, we present main approaches that enable packet propagation in Delay Tolerant Networks (DTNs), explain each one of them, and show which one we used. Then, we present leading Decentralized Social Networks, and discuss whether they could have been used in our scenarios, showing that they actually cannot, and finally present the pinnacle: Grassroots Social Networks. Grassroots Social Networks lay at the core of this research work, and the networking layer presented here is a core work that lays at its foundation.

2.1. Proximate Peer-to-peer Communication Technologies

To create the peer-to-peer communication platform, different technologies were evaluated based on the following criteria:

- **C1 Spread of the protocol:** As a user-facing tool, the communication medium had to be present in virtually all smartphones used today; thus, technologies that are supported by a handful of flagship smartphones, while being an interesting point for future work, are out of the question for this work.
- **C2 Low power:** the communication protocol should not use a high amount of power to propagate packets
- **C3 Target:** the communication protocol should be able to carry text messages efficiently, so a text message should be received within a short time in close range (e.g. 200KB in 1 second).
- **C4 Mobility:** protocols need to support a high node mobility stemming from the mobility patterns of the users
- **C5 Infrastructure:** there can be no external infrastructure necessary for the normal operation of the technology
- **C6 The protocol should be operational even when the app is backgrounded**

2.1.1. Wi-Fi

To relay information directly between peer smartphones, the omnipresent Wi-Fi protocol was first investigated. Propelled by the Wi-Fi Alliance, WLAN is the wireless 802.11 protocol which enables most of the world's computers to communicate. Together with cellular-based mobile networks like 5G, home and commercial routers provide wireless radio access networks for mobile devices to connect to the Internet backbone through a multitude of devices: antennas (Radio Units), computers for signal processing (Distributed Units), base stations (Centralized Units), and more [30]. For the standard Internet-backbone to function, all the aforementioned infrastructure units need to have a constant stream of power, an unblocked radio spectrum, and operational staff. In emergency scenarios such as natural disasters, wars, or politically-motivated violence against the freedom of speech through censorship, media control, or monitoring, the aforementioned infrastructure might not be available for usage. As alternatives to the classical Wi-Fi WLAN technology to facilitate DTNs, we evaluated the following p2p communication media based on the aforementioned common criteria: Wifi Adhoc, Wifi Direct, Wifi Aware, Bluetooth BDR, Bluetooth Low Energy. The Wi-Fi Ad-Hoc mode was the first design to adapt the WLAN standard to direct peer-to-peer wireless communication. However, it suffered from several notable architectural shortcomings, including inefficient power management, a lack of robust security mechanisms, and the requirement for devices to manually decide whether to act as the access point or a client [31]. In the Wi-Fi Ad-Hoc mode, idle smartphones suffer from battery drain due to a lack of distributed power saving mode [29]. However, actively communicating smartphones had the same power cost as using a standard access point, and comparing to Bluetooth, Wi-Fi Ad-Hoc still has a vastly superior throughput rate, which is optimal for larger file transfers. In our context it is still not as useful due to **C3**: the transfer size is minimal, and we optimize for efficient battery usage through **C2**. Wi-Fi Ad-Hoc was eventually replaced by Wi-Fi Direct, which modernized peer-to-peer connectivity by adding better security and an autonomous negotiation protocol that allows connecting devices to automatically determine which one takes over the role of the access point [32, 33]. However, Wi-Fi Direct has two main limitations: it does not support device discovery in the background if the utilizing application is stopped or not actively running in the foreground; another significant constraint of Wi-Fi Direct on modern mobile platforms is that it is not supported by iOS devices, as Apple developed a competing standard, AirDrop [34]. The Android equivalent of AirDrop, QuickShare, is developed cooperatively by Samsung and Google, and since November 2025 supports platform interoperability between Android and iOS, and has had support transfers between Androids, Windows computers, and Chromebooks before. While similarly providing superior data rates, QuickShare, as it uses a combination of Wi-Fi Direct and Bluetooth, is still not usable for this work, as its core lies with Wi-Fi Direct, which requires more power and devices to have the using app constantly foregrounded. Wi-Fi Aware (also known as Neighbor Awareness Networking / NAN) overcomes the background limitation of Wi-Fi Direct by utilizing a highly power-efficient *discover-before-connect* paradigm. Unlike Wi-Fi Direct, Wi-Fi Aware allows devices to automatically organize into synchronized local clusters and use a publish-and-subscribe mechanism to discover nearby services and exchange lightweight messages in

the background, all without establishing a formal network connection. Because it operates without maintaining an active connection state, it has a significantly lower power and memory footprint. If devices need to share large amounts of data, they can seamlessly upgrade their discovery session into a fully bi-directional, high-throughput network connection. Positioning itself as the most appropriate for our use-case, Wi-Fi Aware seems like a promising candidate; however, it was primarily intended for IoT devices [35], support for it on iOS was only recently introduced [36], and thus it was deemed not widely supported enough. Early research into ad hoc and delay-tolerant networks frequently relied on legacy 802.11b and 802.11g standards, which were highly capable for active data transfers but suffered from severe battery drain when devices were left idle [37]. In later standard, 802.11n has introduced a significantly higher throughput and a 17% improvement in energy efficiency over previous standards [29].

2.1.2. Alternatives

With Wi-Fi standards being exhausted for picking the base communication protocol, we evaluated other protocols such as Zigbee and LoRA, but very quickly forsaken due to lacking support for direct communication between mobile phones: Zigbee, LoRA, and 6LowPAN are all intended for small IoT devices to communicate among themselves or with centralized servers [35].

2.1.3. Bluetooth

A technology that is ubiquitous to all smartphones that have been released in the last 10 years is Bluetooth. Being standardized as Bluetooth 1.0 in 1999 by the Bluetooth Special Interest Group (SIG), Bluetooth was introduced to mobile phones by Ericsson in 2001 with the release of the T36, which allowed phones to replace bulky cables transfers with a wireless connection. Since then, Bluetooth has seen four major version bumps:

- Bluetooth 2.0 offered in comparison to Wi-Fi a much lower energy consumption for transferring large files, and higher resilience to signal interference from nearby transmissions.
- Bluetooth 3.0 was introduced as an upgraded specification designed to improve upon the throughput limitations of Bluetooth 2.x, doubling both send/receive throughput speeds to almost 200 KB/s, while the increasing the overall power consumption [29].
- Bluetooth 4.0 represented an elementary shift in Bluetooth technology, with the focus shifting to optimizing for ultra-low power consumption: Bluetooth Low Energy, or BLE, was introduced in this version. BLE presents until today the backbone for most modern smartphone p2p applications, due to major power-saving capabilities: because BLE supports background advertising and discovery without requiring apps to be actively foregrounded, BLE is the primary transport mechanism for decentralized networks and emergency systems like firechat, Briar, Corona-Warn-App, Twimight, BitChat, and more [26, 27, 38, 18].

BLE is characterized by a dual-role split: advertising, which is carried out by the peripheral device, and scanning, which is done by the central device. The central device controls the connection, while the peripheral provides the service. Advertising happens on a set window, in which the peripheral sends out advertisements on an interval. The advertisements contain a customizable device unique identifier, as well as custom manufacturer-specific data. The BLE advertised services stem from the underlying General Attribute Protocol, or GATT. The GATT protocol is a hierarchical protocol, where the highest level is GAP: General Attribute Profile. A GAP groups GATT services together, to offer a specific function, e.g. a heart rate monitor, or a grassroots networking profile. Furthermore, GATT services group a set of characteristics, which are used for interfacing with data, and controlling settings pertaining to the service. Each of the characteristic is identified by a UUID, a value, and has different read/write permissions; the data-flow between the peripheral and the central is bidirectional, with the peripheral being able to notify the central of any changes to a characteristic value as it changes, or central can request to read the current value of a characteristic, and more [39]. To see the peripheral's advertisements, the central device also has to scan during the same time: the scanning is also limited to a window, during which the central listens to all incoming advertisements from surrounding devices. To discover GATT services, a device first must perform a connection to one such a device, which potentially includes pairing - depending on the permissions the device's GATT server defined. BLE presents advantages not only in its conservative use of battery power, but also in its security, and usage model. When an app registers a GATT server with the BLE adapter, the mobile phone controls it natively, and the app can remain backgrounded or on standby, while the OS continues to scan and advertise for services. This allows for both a heightened security model due to the protocol abstraction by the operation system, as well as the grouping together of functionalities from different apps to a central native coordinator. Providing a throughput of about 230 kbps at a low power usage [40], BLE was able to answer all out criteria C1-C5. Due to the aforementioned benefits of the protocol, we thus chose BLE as the application's main communication protocol.

2.2. Remote Peer-to-peer Communication Technologies

After discussing the choice for a proximate communication protocol, this section continues with elaborating on how different protocols were evaluated to facilitate long-distance communication. The protocols that were evaluated in this section include UDP, TCP, HTTPS, QUIC, and UDX.

2.2.1. Application Protocols: HTTP, HTTPS, QUIC

HTTP and HTTPS are the first protocols that we evaluated, as they lie at the base of every modern web applications, and have been tried-and-tested by many applications. Most web servers started by using HTTP, and in a standard web application scenario with a typical client and server having continuous end-to-end connectivity it would be optimal; however to deploy it in a DTN environment where connectivity is intermittent at best, multiple

auxillary systems would need to be used in order to provide hop-by-hop connectivity. As our goal is to provide connectivity with the least amount of external infrastructure, HTTP and its secure variant HTTP over TLS/TLS (HTTPS) require many subsystems that cannot be guaranteed to be freely available in our scenarios. Additionally, HTTPS requires having one constantly listening TCP socket, plus one live TLS connection per peer, each with crypto state. Android and iOS tend to aggressively kill backgrounded applications and sockets. Recovering the socket connections would cost a whole TCP handshake, and a TLS handshake, which would cost multiple round trips per peer. This shows that recovery for TCP-and-TLS-based protocols is a heavy operation that is very expensive in terms of time-to-first-message after start. QUIC, the transport standardized underneath HTTP/3, was designed to overcome two structural limitations of the TCP+TLS stack: its serialized handshakes and head-of-line blocking. Built on top of UDP, QUIC folds the transport and cryptographic handshakes into a single exchange, establishing an encrypted connection in one round trip, and in zero round trips when resuming a previously cached session; it further multiplexes independent streams over one connection, so that a lost packet on one stream does not stall the others [41]. However, QUIC remains connection-oriented: it maintains per-connection state, negotiates a full TLS 1.3 handshake on every cold start, and, like TCP, must re-establish that state whenever the underlying socket is torn down. In a mobile setting where the operating system aggressively reclaims backgrounded sockets, this coupling of transport and cryptographic state reintroduces the very time-to-first-message penalty we sought to avoid. The first protocol that we consideres for that, which already implements all of the above, is QUIC, or Quick UDP Network Connections [Langley2017]. QUIC was designed with a mobile user in mind, supporting seamless network migration through the usage of Connection IDs to route and identify connections, rather than a source/destination IP:port + protocol 5-tuple combination; multiplexing multiple lightweight streams within a single connection allows to avoid the Head-of-Line Blocking issue, which causes queues of packets to buffer and be held up until the first packet is fully processed, severely limiting the performance, and is further exacerbated through out-of-order delivery; finally, in an effort to make common pairings more efficient, QUIC combines the transport connection establishment handshake with a cryptographic handshake to offer 0-RTT connectino setup, allowing data to be sent directly on the first message to known peers. Since QUIC combines the cryptographic layer (normally TLS) with the network layer, it requires continuous cryptographic processing, which reportedly cost up to 3.5 times more CPU than normal TCP using TLS [41]. This computational exertion quickly drains mobile devices, since they are highly battery-constrained, and their CPU is much weaker than the desktop computers QUIC was designed for. Additionally, one of the main advantages of QUIC is the 0-RTT handshake data delivery, which depends on cached address tokens. These cached address tokens often become invalidated when mobile phones roam, dropping the success rate significantly [41]. However, precisely because it combines so many things that UDP normally does not offer, QUIC became a monolith of a protocol that is mostly designed for stable end-to-end connections such as traditional HTTP (HTTP/3) client-server connections.

2.2.2. Transport-level Protocols: TCP, UDP

The elementary transport protocols that enable QUIC and HTTPS are UDP and TCP respectively: to decouple cryptography from connection maintenance and application state, we decided to focus on the transport protocols that enable application communication. Providing congestion-controlled, reliable, ordered, error-checked streams, TCP, the core transport protocol behind the IP protocol stack, was the first protocol that we investigated. As a congestion-controlled transport protocol, TCP assumes that all transmission errors in its delivery path stem from link congestion in the network; that means, that for intermittent connectivity outages, TCP will engage in congestion-avoidance measures, which dramatically reduce the possible data rate on the link temporarily [42]. Moreover, increasing the number of end-to-end hops in the network connection yields rapidly deteriorating performance with the increase of hops [43]. Additionally, the usage of Network Address Translation (NAT) in order to assign multiple hosts to the same IPv4 address in a network makes it even more difficult to establish a p2p TCP connection without using any external proxies, as the NAT mapping is dependent on a 5-tuple: both peers IP addresses, ports, and protocol [44]. Because TCP maintains a strict connection state, the connection will break entirely any element in the 5-tuple formed by the underlying network changes. Every time a mobile device roams, such as switching from a Wi-Fi network to cellular data, or changing its network attachment point, its IP address changes, forcing the TCP connection to be completely re-established from scratch. To overcome the obstacles caused by node mobility and the ensuing change in Internet addresses, we have decided to focus on the UDP protocol as we expect frequent address changes due to our mobility model.

2.2.3. UDX

The transport-layer protocol that was chosen for the system, UDP, is a lightweight, data-oriented network protocol that allows the users to send datagrams without establishing a session or connection between each other. It prioritizes simplicity, incarnate in a lightweight header size, over error-correcting code, reliability, or packet-ordering which are all present in TCP, which makes it ideal for sending lightweight messages as it does not establish reliable connections as its counterpart. UDP or UDP-based protocols are often used in time-critical applications like online gaming, streaming, and other p2p applications. Due to the focus on simplicity, the resulting trade-off yields a transport-layer that, had it based itself directly on UDP, would have needed to implement error-correction codes, deal with network-level packet-loss, congestion control and flow control on active links. In addition to the transport-level control mechanisms, an encryption layer needs to be implemented on-top, in order to guarantee no adversary could read messages that were not intended for them. The transport we ultimately selected for remote peer-to-peer links is UDX, the reliable stream transport that underpins the Holepunch/Pears networking stack [45]. UDX provides reliable, in-order, multiplexed, and congestion-controlled streams over plain UDP, much like QUIC, but with one deliberate design difference: it performs no handshake and no encryption of its own. As the reference implementation phrases it, UDX has ‘no handshakes, no encryption, no

features' [45], which is precisely what makes it inexpensive to (re-)establish. Because UDX is connectionless, a stream is set up by each side independently creating a stream with a locally chosen identifier and connecting to the remote peer using the stream identifier and the remote socket address. Neither peer waits for an acknowledgement before it may send. The very first packet can therefore already carry application payload, making the transport itself effectively *zero round trip*. Encryption and authentication are provided by separate layers: the Noise suite-based [46] secure public key exchanged is performed after the application-level identity exchange happens, with session keys derived from the identity public keys through a birational map [47]. Crucially, this cryptographic handshake is not paid for directly in the transport layer, which makes it very simple to establish such a connection.

2.3. Opportunistic Networking Systems

2.3.1. Background

From the communication protocols we elaborate to research other existing opportunistic networking systems. MANETs, who predate OppNets, seemed promising to enable ad-hoc communication between mobile devices, requiring devices to forward messages for others in addition to allowing the devices themselves to communicate. Talk about different routing strategies in MANETs? One of the leading examples of MANETs that are still developed to this day, is the BATMAN-adv protocol (Better Approach To Mobile Ad-hoc Networking), by the Freifunk Gesellschaft. In BATMAN, nodes constantly broadcast a hello message to their peers, and peers that receive those messages propagate those further. Peers measure the proportion of hello messages that they receive from a node to estimate their link quality, and broadcast that to their peers as well. Peers then re-broadcast the propagation from the peer with the highest ratio link quality, and thus a routing infrastructure is built iteratively. Opportunistic Networks (OppNets) are a subsystem of DTNs, which are an evolution of legacy MANET systems. In MANET systems there has to be a fully-connected end-to-end path before a message can be sent. OppNets are an instance of the Store-Carry-Forward paradigm, where the obstacle that prevents MANETs from being used, is used as an advantage - node mobility. In Pocket-Switched Networks, an alias to OppNets, peers frequently change their network attachment point due to physical movements, as precisely due to the reason alluded to by their name – the participating devices sit in the users' pockets: their smartphones. In those networks, peers carry messages with them as they move until the opportunity arises to forward the message to another peer [48]. In OppNets, the emphasis lies on the opportunistic establishment of links between peers. Every possible participating user is henceforth referred to as a peer in the OppNet, and the links between them establish opportunistically: that means a peer randomly passing another peer will try to establish a (secure) connection to it. Because OppNets rely on the physical mobility patterns of humans to establish bridges between disconnected clusters (either due to network partitions, physical distance, or lacking infrastructure), OppNets excel in decentralization of communication in areas with intermittent connectivity, such as disaster zones, remote areas, or regions that suffer from state control of

the communication infrastructure [33, 49, 50]. The usage of mobility models to propagate information in a network is especially useful to establish information flows in regions that lack communication infrastructure, such as war zones, and disaster-struck areas.

2.3.2. Routing in MANETs and OppNets

In MANETs, routing algorithms assume that the network topology formed is a single connected component, where all nodes can be reached, and strive to establish and maintain an efficient end-to-end path between source and destination by using control packets to evaluate the route [43, 51]. MANET routing algorithms are usually categorized to either on-demand reactive algorithms, or proactive algorithms where nodes frequently send messages to their neighbors to evaluate their link quality. The most common MANET routing algorithms are listed below:

- *AODV*: the Ad-hoc On-Demand Distance Vector algorithm is a *reactive* protocol that discovers routes upon request of a node to send a message to a peer. It floods a Route Request message (RREQ) across the network: every node that forwards the RREQ add who they received it from, in order to record a reverse path. When the destination receives the first RREQ, it sends a Route Response (RREP) along the reverse path, and nodes that forward it back create an entry in their routing table, so that future requests could follow that path. To maintain the route, nodes that detect failures send back a Route Error (RERR) message, triggering another RREQ sequence [19].
- *OLSR*: the Optimized Link State Routing algorithm is a *proactive* protocol where all nodes create and maintain routing tables according to their own link quality, and focuses on evaluating the closest neighborhood for every node, making use of the notion that not every node needs to flood everything to everyone to build efficient routing tables. OLSR uses a Multipoint Relay (MPR) set of neighboring nodes that are well-connected to every two-hop neighbor, in order to form an efficient flooding backbone across the entire network.
- *Geographic*: use GPS location to restrict flooding done by routing protocols, for example by using building geolocation information and solving the routing problem for buildings [13].

On the other hand, OppNet routing does not make the same assumptions: in OppNet routing, there is no end-to-end path that exists. Whether due to sparse connectivity graphs, intermittent network outages, or high node mobility, there is no instantaneous end-to-path that connects all nodes, and the graph is comprised out of multiple components. Instead of dropping a packet when a path is not available to it, OppNet routing algorithms store those packets at a relaying node, the node carries it and forwards it opportunistically when encountering new peers. There exist a plethora of OppNet routing algorithms, and we will cover the three most famous ones:

- *Epidemic*: A flooding approach where a node floods copies of its messages to every other node it encounters. Achieving high delivery rates by spreading data like an epidemic

disease, it is highly prone to congestion due to the massive amount of bandwidth and buffer space needed for an all-to-all broadcast [37, 51].

- *Spray-and-Wait*: This algorithm limits the number of copies of messages created, by creating a constant amount of copies for every message at an originator. In the first phase of the algorithm, the originator ‘sprays’ copies upon encountering other peers: in every encounter, the originator hands over a half of all the copies it has to every peer, until exactly one copy remains, which shall be only forwarded directly to the destination. The relaying nodes continue with the same methodology, until the final node is reached.
- *PRoPHET* (Probabilistic Routing Protocol using History of Encounters and Transitivity): The PRoPHET protocol is a model-based routing algorithm, which uses stochastics to improve the probability of successful delivery, by recording past contact history between nodes and using nodes with the highest probability of eventually reaching a destination efficiently through their mobility pattern.

In this thesis, we use epidemic routing because it has the highest likelihood to deliver a message, at the cost of network congestion, bandwidth, and buffer space due to the DTN stores at every node buffering messages. That is due to focusing on broadcast dissemination, as our goal is to ensure every message is delivered to every unique node in a network at least once [52]. Cases where such a dissemination model is useful is in emergency situations, where disaster alerts have to be spread to as many people as possible as fast as possible, and protest scenarios where it must be ensured that all protestors receive vital information, such as evidence of police brutality, and coordination efforts. Additionally, to create a research baseline, using an epidemic flooding model is beneficial because it lets researchers compare more advanced routing algorithms to a baseline naïve model, which acts as an upper bound for delivery rates and network congestion. Early examples of OppNets have leveraged the notion of Store-Carry-Forward to offer intermittent connectivity without central communication infrastructures [53], providing wide-scale wildlife tracking during mass migrations [50], as well as voicemail and email services to rural areas in developing countries [20, 54]. On smartphones, one of the most famous and large-scale deployments was Firechat [26], which was broadly used in the 2014 Hong Kong protests, and later in Iran and Iraq. Firechat allowed users to use either direct p2p communication, broadcast a message to all mesh participants, or use IRC-style chatrooms based on proximity. Using Firechat, tens-of-thousands of protestors were able to organize and coordinate supplies for the dissidents during the protests, during a time of Internet blackout scare by the government, as protestors chanted for democracy in the Occupy Central protests. However, FireChat was not meant for this use case, as it was lacking basic encryption, privacy, and resilience [55]. From the technical perspective, FireChat utilized similar WiFi and Bluetooth to forward messages over multiple hops without cellular service using a store-and-forward mechanism, using Apple’s Multipeer Connectivity framework. Focusing on the ensuring free information flow in protest use-case, Bridgefy used a similar tech stack but did support encryption: it was used extensively again in the 2019-2020 Hong Kong protests and during Citizenship Amendment Act protests in India 2019. As

research shows [56], adversaries were able to completely shut Bridgefy down using a single well-crafted message delivered across the mesh, deanonymize users, reconstruct users' social graphs, decrypt and read messages. Other notable academic and practical systems include the Serval Project's BatPhone [57], which enables infrastructure-less VoIP mobile telephony for disaster relief using the BATMAN protocol, and Twimight, an open-source Twitter client featuring a 'disaster mode' that allows tweets to spread epidemically from phone to phone via Bluetooth when traditional networks fail [58]. Building upon years of experience as the CEO of Twitter, Jack Dorsey, together with permissionlesstech, released BitChat in July 2015, a highly secure evolution of the aforementioned platforms. BitChat features a dual-stack transport over both a BLE mesh and the WiFi medium, using decentralized Nostr relays to encrypt and anonymize messages when the user is Internet access is available. To efficiently flood messages across the local mesh without overwhelming the network, BitChat implements an epidemic-like routing protocol, which widely distributes data while using node-specific Bloom filters to quickly identify and reject duplicate packets. Furthermore, by natively integrating robust end-to-end encryption and mutual authentication via the Noise Protocol Framework and Noise key exchange, BitChat solves the glaring privacy vulnerabilities that plagued earlier mesh apps. This advanced, censorship-resistant architecture has already seen significant real-world adoption, proving vital for coordination during large-scale protests in Nepal and state-ordered internet blackouts in Iran and Uganda.

2.4. Decentralized Social Networks

Traditional Centralized Social Networks (CSNs) are highly susceptible to Internet outages, control of the underlying infrastructure to monitoring, surveillance, and censorship due to being regulated or controlled by government organizations. Because all data flows through centralized servers in CSNs, authoritarian regimes can easily force companies by restricting financial access, or order executive shutdowns at the network level [4]. When communication is halted in this manner, human social movements are disrupted, and the fundamental right to freedom of (digital) assembly is eliminated without a visible act of physical repression. Governmental control oftentimes leads to repression of human rights during times of political unrest, as access to such information gives governments more power to crack down on political activists and the general population [2, 3]. CSNs require a large set of infrastructure to support the communication backbone that enables the population access to them: more than 3 billion monthly active users use Facebook [59], and access to the aforementioned infrastructure oftentimes requires governmental cooperation through facilitation with network authorities or ISPs. Twitter (now X), after being acquired by Elon Musk, has undergone a massive change in its content policy, moderation, and workforce — which lead to millions of users who left the platform [9]. As a centralized private online platform, Twitter (and others) are sensitive to (hostile) takeovers that can leave users feeling that the platform no longer represents their values. Despite reaching unprecedented global reach, CSNs are constrained by their fundamental dependency on a single controlling authority, which gives rise to severe societal and privacy challenges. Most notably, the free-to-use, user-as-a-product business

model that traditional CSNs use, make use of user data in order to market personalized ads to unsuspecting users, by using embedded tracking and analytic software. This business model, coined Surveillance Capitalism, exploits user data, online behavior, and original content to provide financial value for the platform, with little to no remuneration for the creators, as users are forced to surrender ownership of their data to third-party operators [60]. Additionally, the platform operators wield the controlling power of information, by providing algorithmic curation and content moderation, thereby creating opaque governance that gives rise to environments which are vulnerable to ideological polarization, in which users have limited agency over their digital lives [9, 61]. To counteract the aforementioned vulnerabilities and business practices, Decentralized Social Networks (DSNs) have evolved to distribute data and control across a variety of different entities, to restore user autonomy, data sovereignty, and improve censorship resistance. Based on recent literature [61], DSNs can be categorized into four primary architectural paradigms, each with their own technical and societal trade-offs, and a minimum set of essential agents (the minimal number of nodes whose removal paralyzes the network) [62]. These platforms diverge in their used data structures, content moderation policies, centralization features, and reliance on infrastructure. The first category is the federated architecture, in which networks distribute control over multiple independently operated servers that communicate over standardized protocols. In the federated architecture, control is held by local operators of the platform, which corresponds to an unlimited, but not universal, set of operating agents [62]. A popular example of this architecture is Mastodon, which is comprised out of distributed home servers which all run the same Mastodon code, that interoperate using a common protocol. Subverting the centralization in common platforms like Facebook, X, and Instagram, Mastodon distributes control across multiple independently operated servers. Users on Mastodon register with a home server to store their profile, settings, and post data, and can follow users from all servers. Following a user on another instance relies on the home server performing the subscription on their behalf, which requires the server admins to enable federation, and the remote instance to not be blocked by the home instance's instance blocking feature [10]. Thus, users on a Mastodon server are governed by an administrator (or a set thereof) that set basic terms of usage, user settings, and form a sort of constitution for using them (i.e. terms of agreement). While preventing a single server from owning the entire network, empirical measurements of Mastodon reveal that the user population is largely skewed: just 10% of instances host almost half of all users, and the top 5% of instances generate nearly 95% of all posts [10]. Additionally, the uncoordinated, voluntary nature of the instance administrators lead to instances being heavily co-located within a small set of Autonomous Systems (ASes), with AWS alone accounting for 30% of all global users: this created massive single points of failure, with an outage affecting just the top 10 of global instances wiping out over 62% of all global posts [10]. The empirical data highlights a gap in federated architecture models: without global decentralized mechanisms that synchronize state (i.e. content replication) and perform load balancing across instances, federated isolated networks remain highly sensitive to data loss caused by infrastructure outages. The second category of DSNs is the peer-to-peer (p2p) architecture, which operates without servers, relaying on its users' end devices to form a distributed overlay. This category

of DSNs has an infinite, universal amount of essential agents, as the network is completely decentralized [62]. In the p2p architecture, data is stored locally on user devices, and is relayed to peers directly upon rendezvous. Examples of the p2p architecture include the Secure Scuttlebutt (SSB) protocol, Nostr, and Briar: the foundation of SSB is secure append-only logs that are synchronized between peers (oftentimes over TCP, sometimes over other protocols i.e. Bluetooth Classic in Manyverse) in a manner similar to ‘personal blockchains’. SSB is connection based, and requires users to actively block users they do not wish to communicate with, since it assumes all-to-all communication: unless limited, the user’s log updates are conferred to all other users upon rendezvous. In comparison, Nostr uses a set of global relays that forward cryptographically signed lightweight messages without maintaining global state, while Briar uses WiFi and Bluetooth Classic for close range interactions, and the Tor decentralized network of onion routers to secure relay information over the Internet [27]. Using online relays or lightweight servers to forward messages, platforms like Nostr and Briar suffer from a critical dependence on third party providers, which renders them mostly useless in Internet outages. Without a centralized ‘permissioned’ administration, optimizing for maximum user autonomy, privacy, and censorship resistance, the p2p network makes content availability depend strictly on peers being online; creating a global aggregated view of all posts to discover community trends or enforce network-wide moderation is impossible [61]. The third architecture of DSNs is the blockchain architecture, in which user interactions like posts and likes are recorded as immutable transactions on a public distributed ledger, verified by miners or validators, and ensuring global consistency and transparency [61]. In the blockchain architecture, the number of essential agents is constrained and is finite, and largely corresponds to the bootstrap nodes [62]. Such blockchain systems inherently solve the problem of global trend discovery and data availability, ensuring data stays available forever, but face a clash with rights such as the right to be forgotten, as due to the chain’s immutability harmful content cannot be deleted once created. Additionally, platforms that use plutocratic, token-based incentives (e.g. Steemit) are often controlled or manipulated by ‘whale’ users: users with a large holding of tokens, that allows them to maximize their posts visibility and reachability [61]. That creates a situation in which rich users (possessing a high amount of ‘coins’) have a higher leverage in the platform, trumping poor users. The ability to purchase the platform coins with traditional currency sparks the notion of a plutocracy, in which great wealth dictates control of the government and public discourse [62]. Finally, the fourth architecture of DSNs is the hybrid architecture: platforms belonging to this category attempt to balance the scalability of centralized systems with the user autonomy of decentralized protocols. Bluesky is a major proponent of this architecture: in Bluesky, the different network components are decomposed into independent sub-components that interact. In Bluesky, user identity is secured via Decentralized Identifiers (DIDs), and can be securely transferred between different Personal Data Servers (PDses); using the AT protocol, user data is heaved above residence at traditional data servers, and is located at the user’s PDS provider, which Bluesky encourages users to run themselves. Using the AT protocol, the user can decide to allow Bluesky access to their data identified by their DID record, which allows a centralized component, the Firehose, to efficiently aggregate and cache their data.

Additionally, content curation and moderation are decentralized and decoupled: users can subscribe to ‘Labeler’ and ‘Feed Generators’ accounts, which respectively customize their safety filters and feed generating algorithms, by pulling post labels from third party accounts (and manually deciding what to do with each label generated), and curating content from accounts the user follows [9]. An additional, a rather new proponent of this architecture is BitChat, which is optimized for censorship resistance using a dual-transport protocol. When the Internet is available, BitChat routes messages globally using the aforementioned Nostr relay network, and falls back to offline BLE mesh [63]. In the latter mode, BitChat uses an Epidemic-like protocol (‘Efficient Gossip with Bloom Filters’) in order to propagate messages to remote peers, ensuring vital communication persists despite unavailability of local Internet infrastructure.

2.5. Grassroots Social Networks

To overcome CSNs, the congestion encumbering mesh flooding-based DSNs, the plutocracy of blockchain-based social networks, and the relay dependency of p2p DSNs, Grassroots Social Networks (GSN) proposes a lean p2p architecture, without relaying on any global infrastructure, third-party relays, or shared global ledgers. GSNs operating strictly on the smartphones of its users, granting total user agency over personal information and social graphs, and thus has a universal, infinite amount of essential agents [62]. Grassroots Social Networks aim to provide an egalitarian, cooperative, and democratic alternative to the aforementioned DSNs and CSNs. Unlike in other platforms, by using the GSN any user agent initiates their own instance, choosing their own cryptographic key pair, letting agents operate independently and in isolation of each other by will, sharing data by choice. GSNs are informally defined by having an agreed constitution (the set of basic governing rules that dictate how agents operate): due to the constitution, each user can participate in an endless amount of independent instances, always producing a valid set of operations and state mutations, so that a coalescing of users into a group always produces a seamless, valid ‘global’ state. This work positions itself as networking layer for a larger Grassroots Social Network, which is currently being researched with Ehud Shapiro (Weizmann Institute, Israel and London School of Economics, UK), Idit Keidar (Technion - Israel Institute of Technology), and Ohad Eitan (Technion - Israel Institute of Technology). One of the bigger challenges in both p2p and grassroots social networks, is devising a key management protocol which is secure against major faults [28]: the loss of a smartphone (state loss), or a private cryptographic key (identity loss) - both of which would typically result in the permanent loss of the user’s digital identity and complete social graph. In [64], Eitan et al. devise a solution to the problem of recovering an account with no available central server to reset a password or restore data. By building on the user’s social graph, identity can be recovered by reaching a supermajority of ‘Identity Custodians’: a designated set of trusted friends specified by the user, that authorize a newly-created public key as the reincarnation of an old, discarded key, preserving all of the user’s friendships. If a user still has their private key, state can be recovered using the user’s friends as ‘State Custodians’,

which uses them to recreate the user's social graph and data using the user's private key. In [65], Shapiro outlines the Grassroots Logic Programs (GLP) concurrent, multiagent logic programming language natively designed to implement serverless grassroots platforms on user smartphones. GLP provides a programming model that solves the historic complexity of concurrent logic programming, as well as the scaling and data-replication overhead of blockchain-based architectures - by enforcing Single-Occurrence and Single-Reader Single-Writer (SRSW) invariants on all logic variables, using this Grassroots Network Layer (GNL) as a networking layer to facilitate splitting writers and readers over different devices, by establishing efficient, direct, and authenticated point-to-point communication channels to transmit variable assignments as single, highly efficient network messages. Concretely, this work provides an overview of the GNL built to support GSN over GLP: a hybrid, dual-stack system supporting two communication protocols. In close proximity, a BLE mesh is used to bootstrap friendships and exchange messages containing text and media. When Internet is available, direct IP communication over UDP is supported, after hole-punching is facilitated using connected friends or BLE as a signaling channel.

3. System Design

In this chapter, we lay out the foundations of the problem at hand, and how did we design a solution to approach it. We begin with our assumptions about the environment, and then detail out individual design goals that were chosen because they correlate with an assumption.

3.1. Environment Assumptions

The operating environment for GSN is versatile: from disaster-struck warzones that lack physical access to the Internet because its operating infrastructure was damaged, to politically oppressive regimes that use various Internet monitoring and controlling techniques to oppress and cull resistance to them, providing secure communications in times of political unrest, and providing an end-to-end secure messaging platform for privacy-aware individuals. GSN is designed to run in an absolute absence of centralized infrastructure or dedicated servers, and thus the physical hardware substrate consists solely of smartphones. Smartphones are characterized by the humans that use them: highly mobile, roaming devices characterized by frequently changing IP addresses, constrained battery life, and limited processing power. The first environment that we decided to operate in is disaster zones, which usually have damaged wide-area Internet infrastructure and backhaul connectivity due to the infrastructure's delicate nature [58, 13, 66]. In those scenarios, only a small majority of nodes (e.g. those with satellite links like Starlink plates) retain access to the global Internet [13]. Due to the terrain fragmentation and partial Internet backhaul connectivity, research propose various OppNet and DTN routing strategies, most notably epidemic routing, to pass critical messages directly to one another until reaching a node that still has a connection. To maximize device life and minimize battery drain, we thus decided to focus on BLE as a main transport medium, and to develop a mesh strategy to maximize the transmission range and overall reachability, using other peers to forward messages. Other than disaster zones, another motivating environment is the issue of providing secure communication in politically oppressive regimes, that still have a functioning Internet infrastructure. The underlying transport layer protocol choice was shaped by this design space: TCP is heavily censored in Iran, by using stateful inspection to detect and block banned domains [5], while UDP is heavily restrained due to being associated with VPN traffic and the QUIC protocol, which is used by major Google platforms like Search, YouTube, and Gmail, and Meta CSNs like Facebook and Instagram. While network anomalies targeting the QUIC protocol were detecting during the January 2026 Internet outage in Iran [5], researchers noted that 4.4% of global Internet users cannot use QUIC because UDP is completely blocked, however most of those users were found to

be corporate network users that are behind restrictive enterprise firewalls, and no single ISP was found to completely block QUIC or UDP [41]. Within these settings, the system is designed to operate under the following constraints: **Intermittent Connectivity:** Physical connections between mobile devices are inherently unreliable, sparse, and stochastic due to human mobility, environment interference, or active network shutdowns. This implicates that disjoint communities must be capable of operating independently of any global coordination in localized networks. Due to the various possible settings in which it can be used, the system was designed with letting the user choice of the right security settings for them, as provides a tradeoff between usability and security and privacy, while the default settings for all features are set to the most-cautious, unless chosen otherwise. As mentioned in the previous chapter, we chose UDP as our underlying transport protocol for the IP Internet relation, using it to establish secure communication channels with remote peers. To establish those channels, a hole needs to be punched in each user's firewall to let data packets in from their counterpart. For that, a communication channel needs to be established to be used as a signaling channel, to coordinate hole punching. Initially using the BLE transport to directly coordinate UDP hole-punching, GNL quickly expanded into facilitating hole-punching through existing network connections with mutual online friends (i.e. edge rendezvous points), which uses the notion of propagating trust along the user's social graph to facilitate connections for friends. Additionally, GNL also supports cloud rendezvous points, which are servers that run the rendezvous points, that any GSN user can run on a server with a static IP address and a permissive firewall policy, that allows external incoming connections. Using cloud rendezvous point allows users to reach remote friends without having any overlap in their GSN social graph or having encountered them physically, as rendezvous points allow the creation and redemption of invite links, which are essentially ephemeral, time-constrained, usage-limited expiring tokens that allow the redeemer to cold-call the link creator, letting the redeemer submit a friend request through one of the creator's rendezvous points (both edge and cloud). Pivoting from a traditional client-server architecture to a p2p architecture was clear from the beginning, as the p2p architecture is much more resistant to censorship and control, while the client-server architecture is also more susceptible to disruptions. Finally, to mitigate online monitoring, we decided to enforce end-to-end encryption using a secure Noise handshake for an encrypted session establishment, rendering all transported text undecipherable ciphertext to an adversary that has intercepted a message.

3.2. Design Goals

This section presents design goals and the philosophical foundation for implementation of the network layer. These design goals were derived from the assumptions in section 3.1, and are reflected throughout the system.

Local over remote: The system should prefer local over remote communication. When possible, GNL must use BLE over IP communication to deliver messages: this means, when a user pair has a link with both the BLE and IP protocols, the layer will choose BLE as a carrier medium for a message. This is because BLE involves less external components and exposes

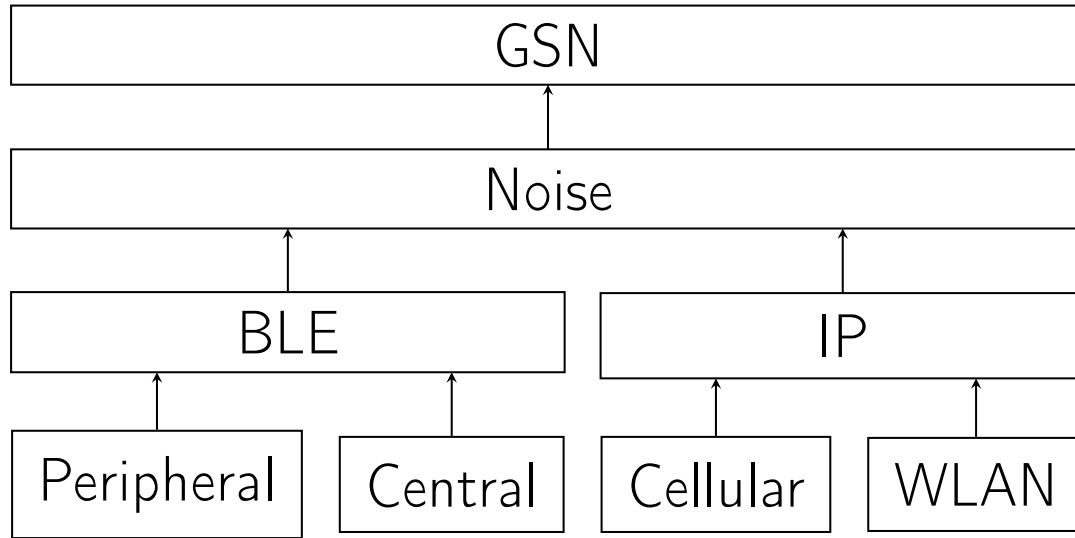


Figure 3.1.: High-level architecture

less information for third parties.

Internet over long distance: When not in close range, the system must use available Internet infrastructure to enable IP communication. When a user pair is befriended, GNL should use IP to convey a message between two remote peers. This means the system will choose to relay a message over the IP medium rather than wait for an opportunistic delivery with the BLE mesh.

Reduce attack interface: The system should expose the minimum amount of required information to strangers in order to facilitate communication. In order to facilitate mesh communication, the cleartext part of the protocol header should be reduced as much as possible. This will harden the system's security guarantees by making it hard to perform profiling attacks, and decrypt ciphertext.

Store-carry-forward: The system should cache packets for unavailable recipients until a possible opportunistic delivery attempt can be made. Digressing from traditional IP routing, the system will try opportunistic delivery of messages for offline peers, queueing messages until a suitable carrier can be found.

User choice: The system must let the user decide on various secure privacy-critical settings. Due to being used in various security profiles affected by different usage scenarios, the system lets the user decide on the usability tradeoff themselves, whether to use sensitive options like a cloud RV point (which is sensitive to control and outages), invite links, IP communication, and more. These options should be configurable in a settings menu.

3.3. Architecture

This section explains the different architectural decisions we've made to implement the networking layer, in order to provide a base communication layer for the Grassroots Social

Networking project. In this section, different design goals are met with environment assumptions, and materialize in actual data structures, code components, networking interfaces, and control triggers. First of all, since GSN targets digital human sovereignty in the modern Internet, we had to pick an Internet-capable device that most humans possess, throughout all of our different usage environments: the smartphone. According to the International Telecommunication Union of the United Nations, mobile phones are the most common gateway to the Internet, with over four in five people worldwide owning an Internet-capable mobile phone [67]. To run GSN on a smartphone, we need to develop a mobile application: for that, multiple programming languages are available, lead by two main paradigms. The first, native application development, would require to develop two parallel versions of the same app, one for iOS written in Swift, and one for Android written in Java. The second paradigm has developers use a language which compiles to both platforms, like Flutter [68] or React Native [69], and develop polyfills on demand for hardware-specific code. As different countries and populations around the world use both iOS devices and Androids, and it is easier to target both platforms while using a compiled language, we decided to use Flutter or React Native rather than developing natively for both platforms, and further decided to use Flutter, as other components of the GSN system were written in Dart [65]. To test out the networking layer, we decided to build a GSN mobile app in Flutter, which would mimic the core functionalities it would offer using GLP; the API document between GLP and GNL is attached as X.X in the appendix.

3.3.1. Identity

The most basic building block of any social network, and more so a GSN, is the user identity. What comprises a user identity in traditional CSNs or DSNs cannot be used in GSNs, as there is no central server to validate a user's email and password credentials. Instead, GSN uses a static per-user cryptographic key pair, which the app generates upon first launch. This key pair is used to sign the user identity periodic announcements (3.3.2), which announce its presence to other GSN users, and used to derive its Noise session key to establish secure encrypted sessions. Messages are encrypted within Noise sessions, and peers that relay them using BLE cannot read any field of the message header, excluding the 'recipientID' field. This field is hashed, concatenating the intended recipient's public key with a static prefix, and a time-slot suffix, so that every peer upon receiving a message can easily calculate whether it is the intended recipient and decrypt the message, or decide to forward it to other peers.

3.3.2. Announce

The Announce message is the only message which is sent in cleartext, and is a neighbor-local message (TTL=1) whose payload carries the full public key, nickname, and the creator's signature over the packet. It is used to broadcast both presence and identity to nearby grassroots peers, and is not relayed over the mesh. Gated on the trust level and the friendship status 3.3.4 of between two peers, a friend receives a more comprehensive version of the announce message than a stranger, containing also candidate IP addresses for a connection.

Announce messages are spread over both delivery substrates, serving different purposes: over the IP transport, the announce message serves as a link healthiness probe, as using UDX sacrifices the ability to know whether a link is actually alive or stale. The transport medium does not influence the shape of the announce message, and it contains the same information.

3.3.3. BLE Advertising & Scanning

As mentioned in previous chapters, the BLE transport medium is used as the chief transport substrate to send and relay messages between grassroots peers. To establish local connections with nearby grassroots devices, a grassroots device makes itself discoverable by registering a GATT server and advertising a BLE service UUID under it. The service UUID has 16 bytes, comprised out of a fixed 8-byte grassroots prefix (the first 8 bytes of SHA-256("grassroots"): the constant 84c403160871e5ad), and a rotating 8-byte suffix comprised out of the first 8-bytes of SHA-256("grassroots ble suffix" | pubkey | slot), where slot is the current wall-clock time divided by a 15-minute slot time. This design allows us to spare costly needless cryptographic trial-decrypt on every message, which in realistic mesh network would result in hundreds of messages per second, which would have paralyze older devices (Cryptobench on Nexus 5X). The trade-off this approach yields is that a recipient public-key is identifiable, but only for a 15-minute slot by adversaries that already have their public key. To a passive observer who does not know the public key, this is a seemingly random beacon that changes every 15 minutes, preventing tracking of a device across slots, while anyone who does know the key - a peer trusted by the owner's trust settings - can quickly compute the suffix and recognize it, quickly deciding whether the intended recipient is addressed to them, otherwise relaying the package forward. The cost of this design materializes in having to match an advertised service UUID against a set of the previous, current, and next derived service UUIDs (due to clock drift) for every known peer, when a user device's trust settings is set to closed, in which a peer blocks connection attempts from any unknown service UUIDs. An open trust setting allows connections from strangers. In order to establish a BLE connection between a device pair, one device must advertise while the other scans; only after an advertisement is seen can a connection attempt be established. Grassroots devices connecting to other grassroots devices are automatically accepted, as the advertised GATT service has the bondless connection option enabled. A scanner that found an advertiser's advertisement attempts to connect to it: if the connection succeeds, the scanner takes on the central role, while the advertiser becomes the peripheral. A connection becomes usable, when the reverse leg also completes: each device is both a central and a peripheral to its counterpart. This is because the performance of a central-side write is faster than a peripheral-side, and it also allows to read the RSSI of the connection.

3.3.4. Trust levels: Friend or Foe?

In GSN, information propagates along the edges of the social graph [70]. That means, that in the default settings of the app, users will not form opportunistic relations with strangers, but only with a determined list of trusted peers, for which they will propagate information.

The setting that controls this is referred to as the trust setting, and two different values allow users to relax that constraint, depending on their usage environment and use case. The trust setting **Closed** has been previously described, and limits new connections a device creates, and its visibility. In the **Closed** mode the device does not perform the announce exchange with strangers, discerning strangers from friends using devised service UUIDs to compare known UUIDs with possible new connections. In order to become friends, users must establish a BLE connection and exchange a bidirectional announce cycle, before sending (and accepting) a friend request. The **Open** trust setting allows the user device to accept incoming connections and correspondingly, tries to establish connection with every grassroots device it finds (prefix-matching of the grassroots constant on advertised service UUIDs), and completes the announce exchange with every such device. A device with an active connection to another can, in addition to friend request and the control-plane announce cycle, exchange basic text and media messages with them in a chat.

3.3.5. Cold-calls, Remote Connection Facilitation: Rendezvous

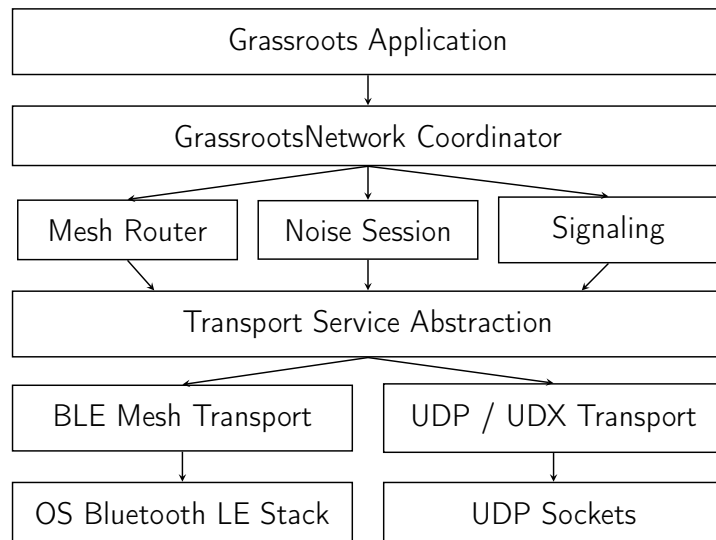
In addition to meeting friends through the BLE substrate, one can befriend grassroots peers through cold-calls: cold-calls are a special mechanism that allows remote peers to create a connection not through previous bootstrapping over BLE, by using either an edge rendezvous point, or a cloud rendezvous point. An edge rendezvous point is actually a friend that has signaled, through a special bit on their announce packet, that they are willing to facilitate new connections with strangers. This allows their peers to create special invite links that can be redeemed at those special facilitator peers. An invite link is limited to a single use, expires in a short time, and requires the facilitator peer to have a permissive NAT and firewall that allows it to accept unsolicited connections ?? . Additionally, an edge rendezvous point can be used as an out-of-band signaling channel to reestablish IP connections by coordinating hole-punching between two NAT'd remote friends, instrumenting them to send punch packets towards their counterpart's observed IP:port NAT mapping. Moreover, any peer can additionally run a cloud rendezvous point, which is a GNL peer that accepts all incoming connections, has a permissive firewall and a static IP address, and is permanently on, and runs the rendezvous protocol. Signaling messages travel in secure Noise sessions, so no stranger could intercept and decrypt them to find out a connection between two (or three) peers exist. The full rendezvous protocol can be observed in the appendix B.7.

3.3.6. Encryption, Key Exchange, Noise Security

In GNL, end-to-end encryption is used to secure messages against observers. To do that, we separate messages to two types: messages that can be encrypted, and messages that can't. The former type, messages that must remain in cleartext, is the announce and the handshake messages; they can't be encrypted because they are necessary to establish an encrypted session (announce by spreading the peer's identity, handshake messages trivially)

3.4. System Architecture

At the code level, BLE exposes separate central and peripheral path IDs for the same peer, while the Noise session manager is keyed by peer identity rather than by transport. The diagram therefore keeps BLE and IP as the two transport substrates and treats Noise as the shared session layer above them.



4. Testbed

4.1. Wireless Opportunistic Networks as a Bridge to Collective Communication

4.1.1. Fundamentals of Collective Communication

Extensive algorithmic and analytical research has established a set of fundamental collective-communication operations: Broadcast, Reduce, All-Reduce, Scatter, Gather, All-Gather and All-to-All [71]. These operations are used to model information flow in collective communication networks, and a generalization of those: parallel algorithms and distributed systems:

- *Broadcast*: The Broadcast operation models a single node sending a copy of its message to all other nodes in the network.
- *Reduce*: One message from each node is combined into a single result using an associative operator such as addition, and delivered to a single node.
- *All-Reduce*: A combination of reduce and broadcast, where the final combined result is then broadcast to all nodes.
- *Scatter*: The Scatter operation models a single node distributing a distinct individual message to all other nodes.
- *Gather*: The Gather operation models all nodes uniting their message to a single, concatenated message, which then gets sent to one node.
- *All-Gather*: A combination of Gather and Broadcast, where the final concatenated result is then broadcast to all nodes.
- *All-to-All*: The All-to-All broadcast has every node broadcast a distinct, individual message to every other node.

To better understand the clash between parallel processing algorithms that require highly-regular interaction patterns with low-level communication mediums (e.g. antennas) present on standard machines, these models were standardized into a set to help research simplify complex parallel computing tasks. The collective communication operations are traditionally modeled and evaluated mathematically across different network topologies, including *line, tree, hyper-cube* topologies. While there is no shortage of analytical results and proven analytical bounds for these operations (e.g. calculating transmission costs), these research

works are mostly constrained to predicted, stable networks and synchronized parallel computing environments. There is a required paradigm shift in adapting these operations to OppNets, as they require mostly synchronous execution over rigid geometric topologies: the research community began mapping those operations to physical mobile devices by using asynchronous delay-tolerant mechanisms: Store-Carry-Forward in all-to-all broadcast epidemic routing, CRDTs for global state synchronization instead of consensus, and federated learning for an all-reduce computation [72, 73]. This testbed evaluates how effectively we can map these collective operations onto the physical world, using an 8-device mesh, simulating different network topologies and movement models. To evaluate the system architecture, we deployed it as a mobile application on a limited testbed of Android and iOS devices. During the deployment, usage statistics were recorded and uploaded to a secure TUM server on an opt-in basis. Before evaluating network-layer behavior, we characterize the radio link itself: at what signal strength a link between two devices can be discovered, established, and used, and what data rate it sustains as it degrades. These two experiments provide the physical-layer baseline that the distances and placements of all subsequent experiments build upon.

4.2. Control plane: RSSI and link establishment on a line

Question. At what received signal strength does a link between two peers become discoverable, established, and usable — and how many bytes does the control plane expose on the air to establish a link and keep it alive?

Setup. Two nodes on stands at fixed height, open field with clear line of sight. The nodes start ~ 120 m apart, beyond radio range, and are brought closer in fixed steps down to 10 m.

Procedure. At each distance step the nodes dwell for several minutes, sampling RSSI and recording link-establishment attempts, before moving to the next step. The sweep is run in both directions — approaching (no link $\rightarrow$ link) and retreating (link $\rightarrow$ drop) — to capture the hysteresis between the signal strength required to *form* a link and that required to *keep* it. We distinguish three link stages, which are expected to occur at different RSSI thresholds:

1. *Discovered*: a peer advertisement (ANNOUNCE) is received;
2. *Established*: the connection and handshake complete;
3. *Usable*: a first payload is delivered and acknowledged end-to-end.

Measures. Per-sample RSSI with ground-truth distance; timestamps of the three link stages per attempt; control-plane bytes on the air (ANNOUNCE, handshake, keepalive traffic) during establishment and during a steady link.

Output. RSSI vs. distance with a log-distance path-loss fit; link-establishment probability and stage breakdown per distance; establishment vs. drop threshold (hysteresis); control-plane cost in exposed bytes to establish a link and to keep it alive per unit time.

4.3. Data plane: throughput of a dilating clique

Question. How does the achievable throughput of a fully-connected clique degrade as the area it covers grows — i.e., how much ground can a clique cover before its capacity collapses?

Setup. Three nodes positioned in an equilateral triangle. The side length is increased step-wise, uniformly dilating the clique from close proximity up to the link-drop distance established in the control-plane experiment.

Procedure. At each side length, first a pairwise single-flow transfer between two nodes establishes the distance-only baseline; then all three pairs run simultaneous transfers (all-to-all workload). Transfers use large, fragmented payloads so that each measurement window captures sustained throughput. The difference between the baseline and the simultaneous runs isolates the cost of medium contention from the cost of distance.

Measures. Per-link RSSI; per-flow goodput; end-to-end latency; delivery failures per side length.

Output. Throughput vs. RSSI and vs. side length, single-flow and contended; aggregate clique throughput vs. covered area, showing how capacity changes as the clique dilates.

5. Evaluation

6. Discussion

7. Conclusion

A. Appendix

B. GLP Networking API Specification

B.1. Introduction

Multiagent deterministic GLP (madGLP) [74] is a distributed runtime for Grassroots Logic Programs (GLP) [75] in which autonomous agents communicate via message passing. The current implementation runs agents as Dart isolates on a single workstation, with an `IsolateManager` routing messages between them [74]. This document specifies the networking API required to replace the `IsolateManager`, enabling madGLP agents to run on separate devices communicating peer-to-peer. The networking API is medium-agnostic: the same interface is used regardless of the underlying communication medium. Two media are specified:

- **BLE** (Bluetooth Low Energy): for communication between physically proximate devices, enabling offline, infrastructure-free operation.
- **IP** (Internet Protocol): for communication between devices at arbitrary distance, using UDX streams over UDP with rendezvous agents for signaling and NAT traversal.

BLE serves as the bootstrap medium where agents first meet face-to-face and establish trust. IP extends that connectivity beyond physical proximity. The networking layer selects the available medium transparently; the madGLP runtime is unaware of which medium carries a given message. The application-level protocols—the social graph [76], friendship, introduction—are implemented entirely in GLP agent code and are not part of this networking specification. To the networking layer, all madGLP messages are opaque payloads.

B.2. Networking API

This section specifies the interface between the madGLP runtime and the networking layer. The interface is independent of the underlying communication medium.

B.2.1. Identity

Each agent has an Ed25519 key pair. The 32-byte public key serves as the agent’s globally unique identity, used for addressing, authentication, and signing. Identity is persistent across sessions.

Function	Signature	Description
Generate identity	<code>putIdentity()</code>	Generate and persist an Ed25519 key pair.

Function	Signature	Description
Retrieve identity	<code>getIdentity() → PubKey</code>	Return the agent's public key.

B.2.2. Peer Discovery and Connection

The networking layer notifies the runtime when peers become reachable or unreachable, regardless of the underlying medium.

When two agents cannot discover each other via BLE proximity, they may exchange identities out-of-band using `generatePeerLink()` and `consumePeerLink(uri)`. A peer link is a signed, time-limited bearer invitation: possession of a valid unused link authorizes one first-contact attempt by the holder.

Function	Signature	Description
Scan for peers	<code>getPeers() → List<Peer></code>	Discover currently reachable peers.
Peer discovered	<code>onPeerDiscovered(cb)</code>	Callback when a new peer is detected. The callback receives the peer's public key and nickname.
Peer connected	<code>onPeerConnected(cb)</code>	Callback when a peer becomes reachable for communication.
Peer disconnected	<code>onPeerDisconnected(cb)</code>	Callback when a peer is no longer reachable.
Check reachability	<code>isPeerReachable(pk) → bool</code>	Whether the given peer is currently reachable via any medium.
Connectivity status	<code>onConnectivityStatus(cb)</code>	Callback when the networking layer detects a change in the agent's public IP address. Triggers the rendezvous protocol (Section B.7). Fired on startup and on address change.
Generate peer link	<code>generatePeerLink() → URI</code>	Generate a signed, one-time, RV-mediated peer-link invitation for out-of-band sharing. Full semantics are specified in Appendix .4.
Consume peer link	<code>consumePeerLink(uri) → PubKey</code>	Verify and redeem an incoming peer-link invitation through a listed rendezvous server, register the inviter's identity, and attempt first contact. Returns the inviter's public key; see Appendix .4.

B.2.3. Cold Calls and Trusted Environment Levels

A *cold call* is a first-contact connection attempt between two agents that do not already share an active grassroots networking communication channel. Cold calls may be initiated by a nearby BLE encounter or by an explicit deep-link shared out-of-band, requiring IP connectivity (Section B.6.2). In the IP case, the inviting agent need not know the redeemer's public key before the link is opened. A cold call is implicitly a friendship request: accepting the cold call establishes friendship.

Agents may enforce one of two cold-call trust levels, affecting BLE encounters:

- **Open:** The agent surfaces unsolicited cold calls from nearby BLE peers as friendship requests for the user to accept or decline.
- **Closed:** The agent ignores unsolicited cold calls from nearby BLE encounters without surfacing them.

Explicit cold-calls through deep-links shared out-of-band are not affected by the different trust levels: the link is itself an explicit invitation by its creator, so the resulting friendship request is always surfaced for the user to accept or decline regardless of the configured trust level. The invite link includes authorization: it contains a temporary, one-time 256-bit token generated by its creator that is consumed upon usage; see Appendix .4 for the whole invite flow.

In the **Open** trust level, an agent responds to any nearby Grassroots BLE encounter by completing the ANNOUNCE exchange, thereby revealing its full public key and allowing first contact. In the **Closed** trust level, an agent does not complete ANNOUNCE with an unknown nearby device: it uses the derived UUID to recognize already-known peers, and ignores unmatched BLE encounters.

B.2.4. Point-to-Point Communication

The networking layer provides reliable delivery of arbitrary byte payloads between peers.

Function	Signature	Description
Send payload	<code>send(pubKey, payload)</code>	Send a byte payload to the peer identified by <code>pubKey</code> .
Receive payload	<code>onReceive(cb)</code>	Register callback for incoming payloads. The callback receives the sender's public key and the byte payload.

B.2.5. Networking Assumptions

The networking layer must satisfy the following assumptions for madGLP correctness: **Fair message delivery:** Every message passed to `send()` is eventually delivered to the recipient,

assuming the recipient eventually becomes reachable. Messages to temporarily-unreachable peers are queued and delivered when connectivity is restored. **Atomic message delivery:** Each message is delivered as a complete unit. If a message is fragmented across multiple packets, the networking layer reassembles them before delivery. **Integrity:** Messages are not corrupted in transit. The networking layer provides encryption and integrity checks. **Authentication:** The sender's identity (public key) is authenticated. A received message from public key pk was indeed sent by the agent holding the corresponding private key.

B.3. madGLP Communication Model

This section summarizes the aspects of madGLP that are relevant to the networking layer. The full specification is in [74].

B.3.1. Global Links and Messages

In madGLP, inter-agent communication is realized through *global links*. A global link connects a local variable pair $(X_p, X_p?)$ at agent p to a local variable pair $(X_q, X_q?)$ at agent q . When the writer at one end is assigned, the value is sent to the other end via a message. All inter-agent messages have the form:

$$G := T^\uparrow$$

where G is a *global variable name* identifying the target endpoint, and $T^\uparrow$ is a *globalized term*—a term in which local variables have been replaced by global names for transmission. A global variable name has the form $_w(p, i)$ or $_r(p, i)$, where p is an agent identifier and $i \in \mathbb{N}$ is an index. The name encodes both the originating agent and the role (writer or reader) of the variable that was globalized.

B.3.2. Message Types

There are two categories of messages, distinguished by the index in the global name: **Regular link messages** (index $i > 0$): A point-to-point assignment sent from the agent where a `global_send` goal fires to the agent that holds the corresponding entry in its global writers table. The message is $G := T^\uparrow$ where G is either $_w(p, i)$ or $_r(p, i)$. Upon receipt, the target agent looks up the entry, assigns the local writer, and removes the entry. **Cold-call messages** (index $i = 0$): A message to an agent's well-known index-0 serializer, used to initiate communication between agents that share no prior variables. The message is $_w(q, 0) := [T^\uparrow \mid _w(q, 0)]$, wrapping the term in a list cell. Upon receipt, the target agent appends the term to its network input stream and updates (but does not remove) the serializer entry. Multiple agents may cold-call the same target concurrently.

B.3.3. Message Flow

The lifecycle of a madGLP message involves three steps, corresponding to three madGLP transactions: **Step 1 — Reduce:** A writer X is assigned at agent p . The paired reader $X?$

becomes known, triggering a `global_send(X?, G, Q)` goal. The goal's reduction globalizes $X?$'s value and adds message $(G := T^\uparrow, Q)$ to p 's outgoing message set M_p . **Step 2 — Send:** The message $(G := T^\uparrow, Q)$ is removed from M_p and placed in the communication channel to Q . *This is the step that the networking layer implements.* **Step 3 — Receive:** The message arrives at agent Q . The runtime looks up G in Q 's global writers table, localizes $T^\uparrow$ (creating fresh local variable pairs for any global names it contains), assigns the local writer, and reactivates any suspended goals. The networking layer is responsible only for Step 2: delivering a serialized message from agent p to agent Q . Steps 1 and 3 are handled entirely within the GLP runtime on each device.

B.4. madGLP-to-Networking Mapping

B.4.1. Agent Identity Mapping

Each madGLP agent identifier (e.g., `alice`, `bob`) is mapped to an Ed25519 public key. The GLP runtime maintains a bidirectional mapping:

- `resolve(agentId) → PubKey`: given a madGLP agent name, return the public key.
- `resolve(pubKey) → AgentId`: given a public key, return the madGLP agent name.

This mapping is established during peer discovery: when agent p discovers agent q via the networking layer, the discovery handshake provides q 's public key, and the runtime registers the association between q 's agent name and public key. For cold-calls, the sender of the network payload must know the target agent's public key. This is obtained either through peer discovery (e.g., BLE proximity) or through out-of-band peer exchange (Section B.2.2). In an IP peer-link cold-call, A creates the signed invite, but the holder B becomes the first network sender: the invite supplies A_{pk} , allowing B to address A 's index-0 serializer.

B.4.2. Message Serialization

A madGLP message $(G := T^\uparrow, Q)$ must be serialized into a byte payload for transmission. The payload encodes two components: the global variable name G and the globalized term $T^\uparrow$.

Global Variable Name Encoding

A global name $_w(p, i)$ or $_r(p, i)$ is encoded as:

Field	Size	Description
tag	1 byte	0x01 for $_w$, 0x02 for $_r$
agent	32 bytes	Ed25519 public key of agent p
index	4 bytes	Index i (big-endian unsigned 32-bit integer)

Total: 37 bytes per global name.

Globalized Term Encoding

A globalized term $T^\uparrow$ is a ground term in which all variables have been replaced by global names. It is a standard term (functors, atoms, integers, lists) that may contain embedded global names. The encoding uses a tagged format:

Tag	Value	Encoding
Atom	0x10	Length-prefixed UTF-8 string
Integer	0x11	8-byte signed big-endian
Global name	0x12	37-byte global name encoding (as above)
Functor	0x20	Tag, length-prefixed name, arity (2 bytes), then each argument recursively
List cell	0x21	Tag, head (recursive), tail (recursive)
Empty list	0x22	Tag only

Message Envelope

The complete serialized message is:

Field	Size	Description
version	1 byte	Protocol version (currently 0x01)
msg_type	1 byte	0x01 for regular link, 0x02 for cold-call (index-0 serializer)
global_name	37 bytes	Target global variable name G
term_length	4 bytes	Length of the serialized term in bytes
term_data	variable	Serialized globalized term $T^\uparrow$

B.4.3. The Send Operation

When the madGLP runtime executes the Send transaction—moving message $(G := T^\uparrow, Q)$ from agent p 's outgoing set M_p to the communication channel—the networking layer performs:

1. **Resolve destination:** Look up the public key for agent Q via `resolve(Q)`.
2. **Serialize:** Encode the message $(G, T^\uparrow)$ into a byte payload according to the format in Section B.4.2.
3. **Transmit:** Call `send(pubKey_of_Q, payload)`.

If the destination agent is not currently reachable, the message is queued locally until the peer becomes reachable. This preserves madGLP's fair message delivery assumption (Section B.2.5).

B.4.4. The Receive Operation

When a payload arrives, the networking layer delivers it to the madGLP runtime:

1. **Identify sender:** The `onReceive` callback provides the sender's public key. Resolve to a madGLP agent identifier via `resolve(pubKey)`.
2. **Deserialize:** Decode the byte payload to recover the global variable name G and the globalized term $T^\uparrow$.
3. **Dispatch:** Pass $(G, T^\uparrow)$ to the madGLP Receive transaction, which performs localization and assignment as specified in the madGLP definition.

B.5. BLE Medium

This section specifies how the networking API (Section B.2) is realized over Bluetooth Low Energy.

B.5.1. BLE Identity and Advertising

Each agent advertises a BLE GATT service whose 128-bit (16-byte) UUID has two parts: a fixed 8-byte *Grassroots prefix* common to all Grassroots agents, and an 8-byte *agent suffix* derived from the agent's public key (e.g., the first 8 bytes of its SHA-256 hash). The prefix enables general discovery of any nearby Grassroots agent; the suffix enables recognition of a specific agent before connection, by anyone who knows that agent's public key.

B.5.2. BLE Discovery

The BLE layer maintains a continuous scan for devices whose service UUID matches the Grassroots prefix. A scan match is only a discovery hint: the advertisement carries neither the agent's public key nor its nickname, so it does not by itself constitute peer discovery. Upon successful BLE connection, an ANNOUNCE packet is exchanged containing the agent's full 32-byte public key and a human-readable nickname. `onPeerDiscovered(cb)` fires upon first receipt of a peer's signed ANNOUNCE — not on a scan match — once per identity (Section B.2.2); `getPeers()` reports the currently reachable peers among those so identified. No Bluetooth bonding or pairing confirmation is required; application-level encryption (using the Ed25519 keys) is used instead.

A BLE encounter with an unknown nearby agent is a local cold-call opportunity. Whether the agent accepts new BLE connections or not is governed by the environmental trust levels in Section B.2.3.

B.5.3. BLE Communication

`send(pubKey, payload)` transmits the byte payload over the established BLE GATT connection. The BLE networking layer handles fragmentation and reassembly for payloads exceeding

the BLE MTU. `onReceive(cb)` delivers reassembled payloads to the runtime.

B.6. IP Medium

This section specifies how the networking API (Section B.2) is realized over the Internet. Communication uses UDP with UDX (reliable multiplexed streams over UDP) for data transport. NAT traversal is handled by the rendezvous protocol (Section B.7).

B.6.1. IP Connection

Once a UDP path is established (either directly or via hole-punching), agents open a UDX stream over it. UDX provides reliable, ordered, congestion-controlled delivery over UDP, satisfying the networking assumptions (Section B.2.5).

Upon establishing the UDX stream, the peers perform a mutual authentication handshake using their Ed25519 keys via the Noise protocol XX pattern (Curve25519 for key exchange, ChaCha20-Poly1305 for encryption, SHA-256 for hashing). The XX pattern transmits each peer's static public key during the handshake, bound to a proof of possession, and derives forward-secret session keys. Each peer then verifies the delivered key against an expected key: the friend's known public key from the social graph for reconnection, or—for an IP cold-call—the inviter's public key from the invite (for the link holder) and the redeemer's public key from the RV-forwarded claim (for the inviter). A mismatch aborts the connection. The X25519 static key carried by Noise is the birational image of the agent's Ed25519 identity key (the standard Ed25519→Curve25519 map), so the expected static is computable from the peer's known Ed25519 public key alone; no separate key distribution is required.

`onPeerConnected(cb)` fires when the UDX stream is established and authenticated. `onPeerDisconnected(cb)` fires when the stream drops.

B.6.2. IP Cold-Call

Over IP, a cold call is bootstrapped by a deep-link sent out-of-band from inviting agent *A* to some holder *B*. The link is a signed bearer invitation: at creation time *A* need not know *B*'s public key, and possession of a valid unused link authorizes exactly one first-contact attempt.

`generatePeerLink()` produces the link and registers the invite with a chosen rendezvous (RV) server. `consumePeerLink(uri)` verifies *A*'s signature, registers *A*'s identity, and submits a cold-call claim to the RV server named in the link. The RV server validates the claim and forwards *B*'s identity to *A*; if *A* accepts the friendship request, the standard rendezvous protocol (Section B.7.4) ensues with the RV server as the signaling agent. Once the UDP path is open, the peers run UDX and Noise XX (Section B.6.1), after which the madGLP cold-call payload is delivered to *A*'s index-0 serializer. Full invite mechanics are specified in Appendix .4.

B.6.3. IP Communication

`send(pubKey, payload)` transmits the byte payload over the authenticated UDX stream. Each message is length-prefixed (4-byte big-endian length followed by payload bytes) to provide message framing. `onReceive(cb)` delivers complete, deframed messages to the runtime.

B.7. Rendezvous

Mobile agents typically run on smartphones behind NAT and do not have static IP addresses. When an agent's IP address changes (e.g., due to network roaming), all its UDP connections break. This section describes how agents re-establish connectivity, using two complementary mechanisms: *rendezvous agents* for bootstrap and *friends-based rendezvous* for subsequent reconnections.

B.7.1. Rendezvous Agents

A *rendezvous agent* is a lightweight GLP agent that runs at a known, static, globally-routable IP address. Its purpose is to coordinate hole-punching between agents that have lost connectivity, and facilitate IP cold-calls. A rendezvous agent:

- Has no friends list and does not participate in the social graph.
- Accepts cold-call connections from any agent.
- Verifies *friendship proofs* (Section B.7.1) to confirm that reconnecting agents are friends, and verifies signed invite capabilities for first-contact IP cold-calls (Section B.6.2).
- Observes each connecting agent's public IP address from the networking layer (via the `peer_address/2` system predicate), serving as an address-discovery mechanism.
- Coordinates UDP hole-punching between the two agents by relaying their addresses.

A machine that hosts a rendezvous agent is called a *GLP rendezvous server*. In the terminology of the ICE framework [77], the GLP rendezvous server combines the roles of STUN server (address discovery via `peer_address/2`) and application-specific signaling server (candidate exchange and hole-punch coordination). It does not serve as a TURN relay; relaying, if needed, is handled at the application level via the social graph (Section B.7.7).

Any individual or organization can run a GLP rendezvous server. The architecture is federated: agents may use multiple rendezvous servers for redundancy, and the system operates without any rendezvous server when agents are BLE-adjacent or share a connected friend.

While connected, agents share with their friends which rendezvous servers they use (and any preferences or priorities) via ordinary GLP messages. Each agent thus knows, for each friend, which rendezvous servers that friend uses. The purpose of this exchange is convergence: the rendezvous protocol is inherently asymmetric — when agent *A*'s IP

changes, both *A* and the disconnected friend *B* must contact the *same* rendezvous agent for the matching to succeed (Section B.7.4). Since *B* cannot reach *A* directly, *B* must know which rendezvous agents *A* uses, and the only time to learn this is while they are connected.

Friendship proof. When two agents establish friendship, each signs a *friendship attestation*: a statement binding the two agents' public keys, signed by the attesting agent's Ed25519 key. An agent can present this attestation to a rendezvous agent to prove that it is a friend of the other agent. The rendezvous agent verifies the signature without needing to know anything about the social graph.

Invite capability. For a first-contact IP cold-call, no friendship attestation exists yet. Instead, the inviter's signed peer link acts as a one-time bearer capability. The link contains a 256-bit nonce, but the nonce is not sent to the rendezvous agent during redemption. The inviter registers an invite identifier and proof key derived from the nonce; the redeemer derives the same values from the link and sends a MAC proof of possession. The rendezvous agent accepts a redemption only if the link signature is valid, the expiration time has not passed, the invite identifier was registered by the inviter, the proof verifies, and the invite is still unused. A successful redemption consumes the invite atomically, and the rendezvous agent then forwards the redeemer's identity to the inviter. Accepting the cold call is accepting the friendship request: the inviter's acceptance establishes friendship and gates the subsequent rendezvous-protocol signaling and Noise handshake.

B.7.2. Friends-Based Rendezvous

A *friends-based rendezvous* occurs when a common friend that is currently connected to both sides coordinates the hole-punch via ordinary GLP messages. Unlike a rendezvous agent, a friend acting in this role does not have a static IP address (smartphone agents never do), and is not dedicated infrastructure—it is simply a friend that happens to be connected to both parties.

Through the social graph, each agent knows the friends of each of its friends. When agents *A* and *B* lose connectivity, each can identify common friends that may still be connected to both sides and request their help.

Friends-based rendezvous is used after bootstrap: once an agent has re-established its first connection (via a rendezvous agent), that reconnected friend can mediate reconnection to further friends. Friends-based rendezvous requires at least one active connection: when all of an agent's connections break (e.g., due to an IP change), it cannot reach any friend to request mediation. A rendezvous agent, with its known static address, provides the initial connection from which friends-based rendezvous can proceed. Alternatively, if two agents are BLE-adjacent, friends-based rendezvous can proceed over BLE without any rendezvous agent — BLE provides the initial connection directly.

B.7.3. Address Maintenance

Each agent's friends list is extended to include IP addresses: for each friend, the agent records not only the communication channel but also the friend's current public address. After reconnection, agents notify their friends of their new address via ordinary GLP messages. This information is maintained as part of the social graph.

An agent behind NAT cannot directly observe its own public address. Instead, it learns its address from the rendezvous agent: when an agent connects to a rendezvous agent, the rendezvous agent observes the agent's public address (the source IP:port of the incoming packets) via the `peer_address/2` system predicate and reports it back. The agent then shares this address with its friends via GLP messages. No external address-discovery infrastructure (such as STUN servers) is needed; the rendezvous agent serves this role.

B.7.4. Rendezvous Protocol

When agent *A*'s IP address changes, all of *A*'s connections break. Reconnection proceeds in two phases: bootstrap via a rendezvous agent, then friends-based rendezvous for the remaining friends.

Bootstrap

Agent *A* detects the IP change via `onConnectivityStatus` (Section B.2.2). Agent *B* detects that *A* went silent via `onPeerDisconnected`. Both *A* and *B* know which rendezvous agents the other uses (learned while they were connected).

1. *A* connects to a rendezvous agent *S* (outbound to *S*'s known static address—always succeeds through NAT). *A* sends a cold-call message: `reconnect(Apk, Bpk, Proof, Reply)`, where *Proof* is *A*'s friendship attestation for *B* and *Reply* is a fresh reply variable.
2. *B* detects that *A* disconnected. *B* sends a UDP packet to each of *A*'s known rendezvous agents, opening a NAT hole so that the rendezvous agent can reach *B*. *B* sends a cold-call message: `available(Bpk, Apk, Reply)`.
3. *S* verifies *A*'s friendship proof, observes *A*'s public address via `peer_address/2` (Section B.7.5), observes *B*'s public address from *B*'s availability message, and matches *A*'s reconnect request with *B*'s availability message. If the proof is invalid, *S* binds *A*'s reply to `rejected` and does not store the request. When the match succeeds, *S* sends *B* a `punch(AddrA)` message via *B*'s reply variable, and sends *A* a `punch(AddrB)` message via *A*'s reply variable.
4. *A* and *B* each receive their `punch(·)` message and call `punch_udp/1` toward the other's observed public address. Each endpoint sends a short burst of UDP packets, opening the corresponding NAT mappings on both sides. Once the punch packets overlap, a direct UDP path is established.

5. The networking layer chooses a deterministic UDX initiator: the endpoint with the lexicographically smaller public key. The initiator opens a UDX stream over the new UDP path, and the other endpoint accepts it. The stream authenticates via Noise XX (Section B.6.1). `onPeerConnected` fires at both agents.

If the first rendezvous agent is unreachable or does not respond within a timeout, *A* and *B* proceed to the next rendezvous agent on their shared list, ordered deterministically (e.g., lexicographically by public key). Since both sides compute the same ordered list, they converge on the same *S*.

Friends-Based Rendezvous

After bootstrap reconnects *A* to one friend *B*, friends-based rendezvous reconnects *A* to the remaining friends. When *A*'s IP changes, all of *A*'s connections break, but the connections among *A*'s friends are unaffected (assuming only *A* changed address). Each agent maintains a friends-of-friends map via the social graph protocol, so every friend of *A* knows which other agents are also friends of *A*.

The protocol proceeds as follows:

1. Bootstrap reconnects *A* to friend *B*.
2. *B* knows (from its friends-of-friends map) which of its own friends are also friends of *A*. *B* immediately performs friends-based mediation for every such common friend *C*: *B* looks up *A*'s and *C*'s addresses in its friends list and sends punch instructions to both sides. No request from *A* is needed.
3. For friends of *A* that are not common friends of *A* and *B*, *A* consults its friends-of-friends map. For each such friend *E*, if *A* has a reconnected friend *D* that is also a friend of *E*, *A* sends *D* a mediation request for *E*. For friends of *A* for which no reconnected friend can mediate, *A* contacts the rendezvous server to reconnect to them.

The process terminates when all of *A*'s friends are reconnected, or when the remaining friends cannot be reached through any available mediator.

A single mediation step works as follows:

1. The mediating friend *D* looks up *A*'s and *C*'s current addresses in its friends list.
2. *D* sends *C*: `punch_to(addrA)`; sends *A*: `punch_to(addrC)`. If *C* is not in *D*'s friends list, the `punch_to` message carries `not_found` and the recipient takes no networking action.
3. Both sides call `punch_udp/1`; the UDX stream establishes.

B.7.5. System Predicates

The rendezvous protocol requires the following system predicates, which provide the interface between GLP agent code and the networking layer:

System Predicate	Description
<code>peer_address(PubKey, Addr)</code>	Unifies <code>Addr</code> with the public IP address and UDP port of the connected peer identified by <code>PubKey</code> , as observed by the networking layer from the source address of incoming packets. Available only for peers with an active connection. Used by rendezvous agents to learn the addresses of the agents they are helping. Also serves as the address-discovery mechanism: an agent behind NAT learns its own public address from the rendezvous agent, which observes it via this predicate.
<code>punch_udp(Addr)</code>	Instructs the networking layer to begin sending UDP packets to <code>Addr</code> to open a NAT mapping.
<code>verify_friendship(Proof A, B, Result)</code>	Verifies that <code>Proof</code> is a valid friendship attestation proving that agents A and B are friends. Binds <code>Result</code> to <code>ok</code> or <code>invalid</code> . Used by rendezvous agents.

All other aspects of the rendezvous protocol—the pending request table on S , the coordination logic, the friends-based traversal strategy, the address maintenance in the friends list—are implemented as ordinary GLP agent code.

B.7.6. GLP Implementation

The rendezvous protocol has been partially implemented and verified in GLP. We describe what has been implemented, what remains, and what is needed to complete the implementation.

What has been implemented. Three components have been implemented in GLP, pass SRSW checking and type checking, and have been verified with comprehensive test suites (presented in full in the appendices):

- **Rendezvous agent** (Appendix .1): the server-side agent that processes cold-call *reconnect* and *available* messages, verifies friendship proofs, observes connecting agents' addresses via `peer_address/2`, maintains a pending request list, and matches requesting pairs. Invalid proofs are rejected via the reply variable. The key mechanism is the *reply-variable chain*: reply variables embedded in cold-call messages are threaded through writer→reader handoffs and assigned when both sides are matched, propagating values back to the original senders.
- **Rendezvous client** (Appendix .2): the client-side building blocks used by agents to construct cold-call messages with embedded reply variables and to dispatch on the replies (*punch* or *rejected*). An end-to-end test wires client and server together on a shared message stream, demonstrating the full bootstrap protocol in both arrival orders.
- **A single friends-based mediation step** (Appendix .3): an extension of the social graph agent in which the friends list is extended to include each friend's current IP address.

The mediating friend uses a fused `lookup_and_send` procedure that, in a single traversal, locates a friend's entry, prepends a *punch_to* message to that friend's output stream, and returns the friend's address. The *mediate* clause calls this procedure twice with cross-references, leveraging GLP's suspension mechanism to resolve the mutual address dependency. Unknown peers are handled gracefully via an *AddrResult* type (*found(Addr)* or *not_found*).

What remains. The friends-based rendezvous protocol (Section B.7.4) relies on a *friends-of-friends map*: each agent must know, for each friend, who that friend's friends are, in order to identify common friends for proactive mediation and to select mediators for explicit requests. This map is not yet implemented in GLP. The friends-of-friends protocol—in which each agent broadcasts friendship updates to all its friends via a shared broadcast stream, and each recipient maintains a local map of observed friendships—has been specified and formally verified (with safety and liveness proofs) in a companion work, but in pseudocode rather than GLP. The current GLP social graph agent tracks only direct friends.

Without the friends-of-friends map, the implemented code covers individual components (the rendezvous agent, the client protocol, and a single mediation step) but cannot drive the full reconnection protocol described in Section B.7.4.

What is needed. To complete the implementation, the following steps are required:

1. **Friends-of-friends map in GLP.** Implement the friends-of-friends protocol as an extension of the GLP social graph agent: each agent broadcasts friendship changes to all friends and maintains a local map of each friend's friends and their status.
2. **Proactive mediation by the bootstrap friend.** When *B* reconnects to *A* (via the rendezvous agent), *B* uses the friends-of-friends map to identify all common friends of *A* and *B*, and proactively mediates for each of them.
3. **Explicit mediation requests.** For friends of *A* not covered by *B*'s proactive mediation, *A* uses its friends-of-friends map to identify a reconnected friend that can mediate, and sends an explicit request.
4. **Integration.** Combine the rendezvous client, the friends-based mediation step, the friends-of-friends map, and the mediation logic into the social graph agent, so that reconnection is triggered automatically when `onPeerDisconnected` fires.

B.7.7. Failure and Fallback

UDP hole-punching succeeds for cone NAT (full-cone, restricted-cone, port-restricted cone) but may fail for symmetric NAT, where the mapped port differs per destination. When hole-punching fails, agents may fall back to relaying messages through the social graph: a connected common friend forwards madGLP messages on behalf of the unreachable peer. Relaying is an application-level concern, implemented as GLP agent code; the networking layer does not relay.

B.8. Public Identity and Access

Some people wish their identity to be publicly known, the way to access them publicly known, or both. To support this, we define two publishable self-certifying records—identity and access—that a person can post or disseminate outside the grassroots platform, e.g. on a personal website, in an email signature, through social media, or via a dissemination protocol such as Nostr or Scuttlebutt. The two records are independent: a person may publish either, both, or neither. Each record can be realized as an executable URL: when opened, the grassroots app adds the person’s identity or access information directly.

B.8.1. Identity Record

An *identity record* binds a person’s public key to public information about them. It consists of:

- the public key of the person (natural or legal);
- the public keys of the person’s identity custodians and the supermajority threshold required for identity replacement;
- any information the person wishes to make public (e.g., first and last name, organizational affiliation);
- if this record replaces or updates a prior identity record: a hash of the public record of the duly-signed identity replacement protocol by the identity custodians of the previous identity;
- a signature by the private key of the person.

Identity custodians, the supermajority threshold, and the identity replacement protocol are defined in a companion paper.

B.8.2. Access Record

An *access record* binds a person’s public key to the rendezvous servers they use. It consists of:

- the public key of the person;
- the addresses of the rendezvous servers the person uses;
- an epoch number;
- a signature by the private key of the person.

Access records carry no expiration. A later record supersedes an earlier one by epoch number: a recipient holding multiple records signed by the same public key retains only the record with the highest epoch number. To change the list of rendezvous servers, the person publishes a new access record with a higher epoch number.

B.8.3. Publication Choices

Publishing only an identity record makes a person publicly identifiable but exposes no information about where they can be reached. Publishing only an access record makes a person reachable via the rendezvous mechanism of Section B.7 to anyone who knows their public key, without revealing any additional information about them. Publishing both makes the person both publicly identifiable and reachable. Publishing neither keeps the person publicly unidentifiable and publicly inaccessible; they remain reachable through friend-mediated introduction, BLE adjacency, and private sharing of their public key and rendezvous servers.

B.9. Complete API Summary

The following table summarizes the complete networking API. This is the interface that replaces the Dart `IsolateManager`. All functions in Section B.2 are medium-agnostic. The system predicates in Section B.7.5 support the rendezvous protocol (Section B.7).

Category	Function	Direction	madGLP Role
Identity			
Setup	<code>putIdentity()</code>	Runtime → Network	Generate agent key pair
Query	<code>getIdentity()</code>	Runtime ← Network	Retrieve agent public key
Discovery			
Scan	<code>getPeers()</code>	Runtime → Network	Find reachable agents
Notify	<code>onPeerDiscovered(cb)</code>	Runtime ← Network	Register agent-to-pubkey mapping
Connect	<code>onPeerConnected(cb)</code>	Runtime ← Network	Agent is now reachable
Disconnect	<code>onPeerDisconnected(cb)</code>	Runtime ← Network	Agent no longer reachable; queue messages
Reachable	<code>isPeerReachable(pk)</code>	Runtime → Network	Check before send
Connectivity	<code>onConnectivityStatus(cb)</code>	Runtime ← Network	Report IP address changes
Gen link	<code>generatePeerLink()</code>	Runtime → Network	Create signed RV-mediated invite URI
Use link	<code>consumePeerLink(uri)</code>	Runtime → Network	Redeem invite through RV and import inviter
Communication			
Send	<code>send(pubKey, payload)</code>	Runtime → Network	Implements madGLP Send transaction

Category	Function	Direction	madGLP Role
Receive	onReceive(cb)	Runtime $\leftarrow$ Network	Feeds madGLP Receive transaction
System Predicates (Section B.7.5)			
Peer address	peer_address(PubKey, Addr)	GLP $\leftarrow$ Network	Observe connected peer's public address
UDP punch	punch_udp(Addr)	GLP $\rightarrow$ Network	Open NAT mapping for hole-punching
Verify	verify_friendship(Proof, A, B, Result)	GLP $\leftarrow$ Network	Verify friendship attestation

B.9.1. Correspondence to IsolateManager

The following table shows how each IsolateManager responsibility maps to networking calls:

IsolateManager	Networking	Notes
Route message to target isolate	send(pubKey, payload)	Resolve agent name to pubkey, serialize, transmit
Deliver to isolate's incoming stream	onReceive(cb)	Deserialize, dispatch to Receive transaction
Register isolate for agent	onPeerConnected(cb)	BLE/UDX connection replaces isolate spawn
Cold-call to index-0 serializer	send(pubKey, payload)	Same primitive; msg_type field distinguishes regular from cold-call
Global link assignment delivery	send(pubKey, payload)	Same primitive; msg_type field distinguishes

.1. Rendezvous Agent — GLP Implementation

The rendezvous agent processes cold-call messages on its input stream. A *reconnect* message from agent *A* includes *A*'s public key, friend *B*'s public key, a friendship proof, and a reply variable. An *available* message from *B* includes both public keys and a reply variable.

The agent verifies the friendship proof via `verify_friendship` and observes each connecting agent's public address via `peer_address` (both body system predicates). It searches a pending list for a matching request from the other side. If found, `complete_rendezvous` constructs the reply values in its head: *A* receives `punch(AddrB)` and *B* receives `punch(AddrA)`. If the proof is invalid, the reply is bound to *rejected*. If no match is found, the request is stored in the pending list.

The key GLP mechanism is the *reply-variable chain*: each cold-call message contains a reply variable (a writer held by the sender). As the rendezvous agent processes the message, the reply variable is threaded through writer $\rightarrow$ reader handoffs. When both sides are matched, the assignments propagate back through the chain to the original senders.

```

SignalReply ::= punch(Constant) ; rejected.
VerifyResult ::= ok ; invalid.

SignalMsg ::= reconnect(Constant, Constant, Constant, SignalReply?)
            %% reconnect(A_pk, B_pk, Proof, Reply)
            ; available(Constant, Constant, SignalReply?).
            %% available(B_pk, A_pk, Reply)

PendingEntry ::= needs(Constant, Constant, Constant, SignalReply?)
              %% needs(A_pk, B_pk, AddrA, ReplyA?)
              ; ready(Constant, Constant, Constant, SignalReply?).
              %% ready(B_pk, A_pk, AddrB, ReplyB?)
PendingList ::= [] ; [PendingEntry | PendingList].

procedure rv_agent(Stream(SignalMsg)?, PendingList?).

rv_agent([reconnect(A, B, Proof, ReplyA?)|In], Pending) :-
    ground(A?), ground(B?) |
    verify_friendship(Proof?, A?, B?, Status),
    peer_address(A?, AddrA),
    handle_verify(Status?, A?, B?, AddrA?, ReplyA, Pending?, Pending1),
    rv_agent(In?, Pending1?).

rv_agent([available(B, A, ReplyB?)|In], Pending) :-
    ground(A?), ground(B?) |
    peer_address(B?, AddrB),
    find_needs(A?, B?, AddrB?, ReplyB, Pending?, Pending1),
    rv_agent(In?, Pending1?).

rv_agent([], _).

procedure handle_verify(VerifyResult?, Constant?, Constant?, Constant?,
                       SignalReply, PendingList?, PendingList).
handle_verify(ok, A, B, AddrA, ReplyA?, Pending, Pending1) :-
    find_ready(A?, B?, AddrA?, ReplyA, Pending?, Pending1).
handle_verify(invalid, _, _, _, rejected, Pending, Pending?).

procedure find_ready(Constant?, Constant?, Constant?, SignalReply,
                    PendingList?, PendingList).
find_ready(A, B, AddrA, ReplyA?, [ready(B2, A2, AddrB, ReplyB?)|Ps], Ps?) :-
    A? == A2?, B? == B2? |
    complete_rendezvous(AddrA?, AddrB?, ReplyA, ReplyB).
find_ready(A, B, AddrA, ReplyA?, [P|Ps], [P?|Ps1?]) :-
    otherwise |
    find_ready(A?, B?, AddrA?, ReplyA, Ps?, Ps1).
find_ready(A, B, AddrA, ReplyA?, [], [needs(A?, B?, AddrA?, ReplyA)]).

procedure find_needs(Constant?, Constant?, Constant?, SignalReply,

```

```

        PendingList?, PendingList).
find_needs(A, B, AddrB, ReplyB?, [needs(A2, B2, AddrA, ReplyA?)|Ps], Ps?) :-
    A? =?= A2?, B? =?= B2? |
    complete_rendezvous(AddrA?, AddrB?, ReplyA, ReplyB).
find_needs(A, B, AddrB, ReplyB?, [P|Ps], [P?|Ps1?]) :-
    otherwise |
    find_needs(A?, B?, AddrB?, ReplyB, Ps?, Ps1?).
find_needs(A, B, AddrB, ReplyB?, [], [ready(B?, A?, AddrB?, ReplyB)]).

procedure complete_rendezvous(Constant?, Constant?, SignalReply, SignalReply).
complete_rendezvous(AddrA, AddrB, punch(AddrB?), punch(AddrA?)).

```

The system predicates `peer_address/2`, `verify_friendship/4`, and `punch_udp/1` are described in Section B.7.5.

Open: cold-call branch not yet supported. The code above handles only the friend-reconnect form of reconnect: every reconnect message is fed unconditionally to `verify_friendship/4`, which rejects anything that is not a valid friendship attestation. The IP cold-call flow (Appendix .4, step 5), however, has *A* send a reconnect whose Proof slot carries an invite-ID reference to the pre-registered invite rather than a friendship attestation; the rendezvous agent is expected to dispatch on the proof variant and run a different verifier. Two candidate resolutions, neither applied here:

- Make the Proof type polymorphic (friendship attestation vs. invite-ID reference) and have `rv_agent` dispatch on the variant before calling the appropriate verifier.
- Introduce a dedicated `cold_call_accept` message type with its own handler, leaving reconnect untouched.

The code in this appendix is left as-is pending that choice.

.2. Rendezvous Client — GLP Implementation

The client side provides building blocks for agents that need to use a rendezvous agent. `make_reconnect` constructs a *reconnect* cold-call message with a fresh reply variable embedded, returning the message and the paired reply reader. `make_available` does the same for *available* messages. `handle_reply` dispatches on the reply: `punch(Addr)` triggers `punch_udp`; *rejected* requires no networking action.

```
SignalReply ::= punch(Constant) ; rejected.
```

```

SignalMsg ::= reconnect(Constant, Constant, Constant, SignalReply?)
            %% reconnect(A_pk, B_pk, Proof, Reply)
            ; available(Constant, Constant, SignalReply?).
            %% available(B_pk, A_pk, Reply)

```

```

procedure make_reconnect(Constant?, Constant?, Constant?,
                        SignalMsg, SignalReply).
make_reconnect(A, B, Proof,
               reconnect(A?, B?, Proof?, Reply),
               Reply?).

procedure make_available(Constant?, Constant?, SignalMsg, SignalReply).
make_available(B, A,
               available(B?, A?, Reply),
               Reply?).

procedure handle_reply(SignalReply?).
handle_reply(punch(Addr)) :- ground(Addr?) | punch_udp(Addr?).
handle_reply(rejected).

```

An end-to-end test wires the client and server together on a shared message stream:

```

procedure test_full_scenario(SignalReply, SignalReply).
test_full_scenario(ReplyA?, ReplyB?) :-
    make_reconnect(alice, bob, proof, MsgA, ReplyA),
    make_available(bob, alice, MsgB, ReplyB),
    rv_agent([MsgA?, MsgB?], []).

```

After execution, *ReplyA* is bound to *punch(stub_addr_bob)* and *ReplyB* to *punch(stub_addr_alice)*.

.3. Friends-Based Rendezvous — GLP Implementation

Friends-based rendezvous is implemented as an extension of the social graph agent. The friends list is extended to include each friend's current IP address: entries have the form *fe(Name, OutputStream, Addr)*. Agents notify friends of address changes via *addr_update* messages.

When agent *A* asks friend *B* to mediate reconnection to *C*, the procedure *lookup_and_send* performs a fused lookup-and-send: in a single traversal of the friends list, it locates a friend's entry, prepends a *punch_to* message to that friend's output stream, and returns the friend's address. The *punch_to* argument carries an *AddrResult: found(Addr)* if the peer was located, or *not_found* if not. The recipient dispatches accordingly — *punch_udp* is never called with an invalid address.

The *mediate* clause calls *lookup_and_send* twice with cross-references: the first call sends *A* a *punch_to(AddrC?)* where *AddrC* is not yet bound; the second call binds *AddrC* when it finds *C*'s entry. GLP's suspension mechanism fills in the unbound reader once the writer is bound — the standard futures/promises mechanism.

```

AddrResult ::= found(Constant) ; not_found.

MsgOp ::= mediate(Constant)          %% mediate(C_pk)

```

```

; punch_to(AddrResult)      %% punch_to(found(Addr)) or punch_to(not_found)
; addr_update(Constant).    %% addr_update(NewAddr)

Msg ::= msg(Constant, Constant, MsgOp).
MsgStream ::= [] ; [Msg | MsgStream].
FriendStream ::= [] ; [Msg | FriendStream].
FriendEntry ::= fe(Constant, FriendStream?, Constant).
FriendList ::= [] ; [FriendEntry | FriendList].

procedure fmrsv(Constant?, MsgStream?, FriendList?).

fmrsv(Id, [msg(From, To, addr_update(Addr))|In], Fs) :-
    ground(Id?), Id? == To? |
    update_addr(From?, Addr?, Fs?, Fs1),
    fmrsv(Id?, In?, Fs1?).

fmrsv(Id, [msg(From, To, mediate(C))|In], Fs) :-
    ground(Id?), ground(From?), ground(C?), Id? == To? |
    lookup_and_send(From?, Id?, AddrC?, AddrFrom, Fs?, Fs1),
    lookup_and_send(C?, Id?, AddrFrom?, AddrC, Fs1?, Fs2),
    fmrsv(Id?, In?, Fs2?).

fmrsv(Id, [msg(_, To, punch_to(found(Addr))|In], Fs) :-
    ground(Id?), Id? == To? |
    punch_udp(Addr?),
    fmrsv(Id?, In?, Fs?).

fmrsv(Id, [msg(_, To, punch_to(not_found))|In], Fs) :-
    ground(Id?), Id? == To? |
    fmrsv(Id?, In?, Fs?).

fmrsv(_, [], _).

procedure lookup_and_send(Constant?, Constant?, AddrResult?, AddrResult,
                          FriendList?, FriendList).
lookup_and_send(Key, Id, OtherAddr, found(Addr?),
    [fe(K, [msg(Id?, K?, punch_to(OtherAddr?))|Out1?], Addr)|Rest],
    [fe(K?, Out1, Addr?)|Rest?]) :-
    Key? == K?, ground(Addr?) | true.
lookup_and_send(Key, Id, OtherAddr, KeyAddr?,
    [F|Rest], [F?|Rest1?]) :-
    otherwise |
    lookup_and_send(Key?, Id?, OtherAddr?, KeyAddr, Rest?, Rest1).
lookup_and_send(_, _, _, not_found, [], []).

procedure update_addr(Constant?, Constant?, FriendList?, FriendList).
update_addr(Key, NewAddr, [fe(K, Out?, _)|Fs],
    [fe(K?, Out, NewAddr?)|Fs?]) :-

```

```

    Key? == K? | true.
update_addr(Key, NewAddr, [F|Fs], [F?|Fs1?]) :-
    otherwise | update_addr(Key?, NewAddr?, Fs?, Fs1).
update_addr(_, _, [], []).

```

.4. IP Cold-Call Invite Redemption

An IP cold-call peer link is a signed bearer invitation created by agent *A* for an as-yet-unknown holder *B*. The link is base64url-encoded and is redeemed through a rendezvous server.

```

Invite ::= invite(
    A_pk,
    A_nickname,
    RvServer,      %% RV address and public key
    Nonce256,
    ExpiresAt,
    SignatureA
).

```

Nonce256 is 256 uniformly random bits. ExpiresAt is one hour after creation. RvServer contains a rendezvous server’s public address and key. SignatureA is an Ed25519 signature by *A* over all preceding fields. The nonce is usable exactly once: *A* records the invite as unused with the rendezvous agent when the link is generated, and the rendezvous agent marks it consumed atomically when validating the redeemer’s claim. Possession of the link authorizes a first-contact attempt; friendship is established only when *A* accepts the resulting cold call from *B*.

The link is base64-encoded for compact URI transport, allowing the recipient to open it as a deep-link. The protocol relies on the out-of-band channel to deliver the link to the intended recipient, and relies on the 256-bit nonce, one-hour expiration, and *A*’s signature to prevent guessing, replay, and tampering.

Both *A* and any holder of the link derive an opaque invite identifier and proof key:

$$\begin{aligned}
 InviteId &= H(\text{"glp invite id"} \parallel Nonce256), \\
 InviteKey &= HKDF(Nonce256, \text{"glp invite proof"} \parallel A_{pk} \parallel RvServer \parallel ExpiresAt).
 \end{aligned}$$

The raw nonce is not sent over the IP network. Network messages use InviteId plus MACs under InviteKey, so passive observers and off-path attackers cannot learn or guess the nonce from redemption traffic.

.4.1. Rendezvous-Gated Redemption

1. **Generation and registration.** *A* picks a fresh 256-bit secret, computes the invite ID and proof key from it, and assembles the link (public key, nickname, the chosen rendezvous server *S*’s address and public key, secret, expiration). *A* signs the link with its private

key and records the invite locally as unused until the expiration. Over *A*'s existing authenticated connection to *S*, *A* registers the invite by sending *A*'s public key, the invite ID, the proof key, and the expiration. *S* stores these together with an "unused" flag.

2. **Link delivery.** *A* sends the base64url-encoded link to *B* out-of-band.
3. **Cold-call claim.** *B* opens the link. `consumePeerLink(uri)` decodes it, verifies *A*'s signature, checks that it is still valid, registers *A*'s public key and nickname locally, and recomputes the invite ID and proof key from the secret. *B* opens an encrypted, authenticated connection to *S* (using *S*'s public key from the link) and submits a cold-call claim: the invite ID, *B*'s public key, *B*'s nickname, and a short proof computed from the proof key covering *A*'s public key, *B*'s public key, the invite ID, and a fresh nonce of *B*'s own. *B* then sends a standard rendezvous available message naming itself and *A*.
4. **Claim validation and forwarding.** *S* checks four conditions: the invite ID is registered for *A*; the expiration is still in the future; the invite is still marked unused; and the proof verifies against the proof key stored in step 1. On any failure, *S* rejects the claim. On success, *S* atomically flips the invite to "used" and forwards *B*'s public key and nickname to *A* over *A*'s existing connection to *S*.
5. **Acceptance.** *A* presents *B*'s public key and nickname to the application layer as a friendship request. If *A* accepts the request (which is the friendship decision), *A* sends *S* a standard rendezvous reconnect message naming *B*. In place of the friendship attestation that the reconnect message normally carries, the proof field references the invite ID that *A* pre-registered with *S* in step 1; *S* matches it against its invite state to authorize the request.
6. **Signaling.** *S* now holds *A*'s reconnect and *B*'s available for the same pair of public keys, and the standard rendezvous protocol (Section B.7.4) takes over with *S* as the signaling agent. *S* observes both public addresses from the connections it holds and sends each side the other's address.
7. **UDP path and Noise handshake.** *A* and *B* each send UDP packets toward the address they received. Once the UDP path is open, the side designated as initiator (by a deterministic rule based on the public keys) opens a UDX stream, and both sides run the Noise XX handshake using their Ed25519 keys (Section B.6.1). The rendezvous server sees signaling metadata and observed addresses, never the encrypted payload.
8. **Cold-call delivery.** Only after the Noise-authenticated UDX stream is established does *B* send the madGLP cold-call payload, which the networking layer delivers to *A*'s index-0 serializer. The friendship between *A* and *B* was established by *A*'s acceptance in step 5.

List of Figures

3.1. High-level architecture 21

List of Tables

Bibliography

- [1] R. Sabet Sarvestany. “Political Violence amid Declining Legitimacy: Patterns of State Repression”. In: *Raptor Journal* (Jan. 2026).
- [2] G. Awwad and K. Toyama. “Digital Repression in Palestine”. In: *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. CHI ’24. Honolulu, HI, USA: Association for Computing Machinery, 2024. ISBN: 9798400703300. DOI: 10.1145/3613904.3642422. URL: <https://doi.org/10.1145/3613904.3642422>.
- [3] A. f. P. C. Derechos Digitales. *Internet shutdowns and human rights*. OCHR Submission. Available at: https://www.apc.org/sites/default/files/internet_shutdowns_and_human_rights_ohchr_submission_2022.pdf. Accessed on: 2025-11-03 21:00 GMT+0200. 2022.
- [4] L. López. “Democracy in an Age of Networked Control”. In: *Journal of Democracy* 37.3 (2026), pp. 143–155.
- [5] G. Aceto, V. Persico, and A. Pescapè. *Iran’s January 2026 Internet Shutdown: Public Data, Censorship Methods, and Circumvention Techniques*. 2026. arXiv: 2603.28753 [cs.NI]. URL: <https://arxiv.org/abs/2603.28753>.
- [6] S Suwal. “The 2025 Gen Z uprising in Nepal: A three-part analysis”. In: *Harvard TH Chan: School of Public Health* (2025).
- [7] E. T. Cluster. *Communication and Information Dynamics in Gaza*. Brief. Available at: https://www.etcluster.org/sites/default/files/documents/Communications_and_Information_Dynamics_in_Gaza_December_2023_1.pdf. Accessed on: 2025-11-03 21:00 GMT+0200. 2023.
- [8] Reliefweb. *Gaza crisis: Update: Internet Connectivity Blackout in Gaza*. Situation Report. Available at: <https://reliefweb.int/report/occupied-palestinian-territory/gaza-crisis-update-internet-connectivity-blackout-gaza-16-june-2025-emergency-telecommunications-cluster-etc>. Accessed on: 2025-11-03 21:00 GMT+0200. 2025.
- [9] L. Balduf, S. Sokoto, O. Ascigil, G. Tyson, B. Scheuermann, M. Korczyński, I. Castro, and M. Król. “Looking AT the Blue Skies of Bluesky”. In: *Proceedings of the 2024 ACM on Internet Measurement Conference*. 2024, pp. 76–91.
- [10] A. Raman, S. Joglekar, E. D. Cristofaro, N. Sastry, and G. Tyson. “Challenges in the Decentralised Web: The Mastodon Case”. In: *Proceedings of the Internet Measurement Conference*. IMC ’19. Amsterdam, Netherlands: Association for Computing Machinery, 2019, 217–229. ISBN: 9781450369480. DOI: 10.1145/3355369.3355572. URL: <https://doi.org/10.1145/3355369.3355572>.

- [11] S. Nakamoto. "Bitcoin: A peer-to-peer electronic cash system". In: *Available at SSRN 3440802* (2008).
- [12] V. Buterin et al. "Ethereum white paper". In: *GitHub repository* 1.22-23 (2013), pp. 5–7. URL: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf.
- [13] Z. Liu, O. Chabra, J. Lynch, A. Martin, C. Li, and H. Balakrishnan. "Scalable Routing in a City-Scale Wi-Fi Network for Disaster Recovery". In: *1st New Ideas in Networked Systems (NINeS 2026)*. Ed. by K. Argyraki and A. Panda. Vol. 139. Open Access Series in Informatics (OASICS). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, 10:1–10:31. ISBN: 978-3-95977-414-7. DOI: 10.4230/OASICS.NINeS.2026.10. URL: <https://drops.dagstuhl.de/entities/document/10.4230/OASICS.NINeS.2026.10>.
- [14] P. H. Kindt, M. Saur, M. Balszun, and S. Chakraborty. "Neighbor Discovery Latency in BLE-Like Protocols". In: *IEEE Transactions on Mobile Computing* 17.3 (2018), pp. 617–631. DOI: 10.1109/TMC.2017.2737008.
- [15] L. Stotts, S. Seidel, T. Krout, and P. Kolodzy. "Manet gateways: radio interoperability via the internet, not the radio". In: *IEEE Communications Magazine* 46.6 (2008), pp. 51–59.
- [16] Wi-Fi Alliance. *Wi-Fi Aware™*. URL: <https://www.wi-fi.org/alternative-topologies>.
- [17] Bluetooth Special Interest Group. *Bluetooth Core Specification Version 4.2*. URL: <https://www.bluetooth.com/specifications/specs/core-specification-4-2/>.
- [18] m. o. s. c. Jack Dorsey. *bitchat*. GitHub. Available at: <https://github.com/permissionlesstech/bitchat/blob/main/WHITEPAPER.md>. Accessed on: 2025-11-03 21:00 GMT+0200. 2025.
- [19] P. T. Jörg Ott Ljubica Kärkkäinen. *MANETs and VANETs*. Lecture notes for Connected Mobility Basics, Technical University of Munich. 2025. URL: <https://cit.tum.de>.
- [20] A. Lindgren and A. Doria. "Experiences from deploying a real-life DTN system". In: *Proceedings of the Fourth IEEE Consumer Communications and Networking Conference*. Las Vegas, Nevada, USA: IEEE, 2007, pp. 217–221. ISBN: 978-1-4244-0667-6. DOI: 10.1109/ccnc.2007.50.
- [21] D. Guzman, D. Trossen, M. McBride, and X. Fan. "Insights on Impact of Distributed Ledgers on Provider Networks". In: *Blockchain – ICBC 2022: 5th International Conference, Held as Part of the Services Conference Federation, SCF 2022, Honolulu, HI, USA, December 10–14, 2022, Proceedings*. Honolulu, HI, USA: Springer-Verlag, 2022, 3–17. ISBN: 978-3-031-23494-1. DOI: 10.1007/978-3-031-23495-8_1. URL: https://doi.org/10.1007/978-3-031-23495-8_1.
- [22] D. Guzman, D. Trossen, and J. Ott. *Multicast Applicability for Distributed Consensus Systems*. Internet-Draft draft-guzman-multicast-applicability-dcs-00. Work in Progress, Expired 4 September 2025. Internet Engineering Task Force, 2025.

- [23] P. Maymounkov and D. Mazières. “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric”. In: *Revised Papers from the First International Workshop on Peer-to-Peer Systems*. IPTPS '01. Berlin, Heidelberg: Springer-Verlag, 2002, 53–65. ISBN: 3540441794.
- [24] D. Guzman, D. Trossen, and J. Ott. “If Iterative Diffusion Is The Answer, What Was The Question?” In: *Proceedings of the 2nd ACM SIGCOMM Workshop on Future of Internet Routing & Addressing*. FIRA '23. New York, NY, USA: Association for Computing Machinery, 2023, 29–34. ISBN: 9798400702761. DOI: 10.1145/3607504.3609288. URL: <https://doi.org/10.1145/3607504.3609288>.
- [25] D. Guzman, D. Trossen, T. V. Doan, and J. Ott. “Proliferation of the Service-centric Distributed Consensus Model and its Impact on Ethereum”. In: *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. ICBC '24. Dublin, Ireland: IEEE, 2024, pp. 1–6. ISBN: 9798350316742. DOI: 10.1109/ICBC59979.2024.10634450. URL: <https://doi.org/10.1109/ICBC59979.2024.10634450>.
- [26] T. Simonite. *The Latest Chat App for iPhone Needs No Internet Connection*. Article on MIT Technology Review. Available at: <https://web.archive.org/web/20151127150930/http://www.technologyreview.com/news/525921/the-latest-chat-app-for-iphone-needs-no-internet-connection/>. Accessed on: 2025-11-03 21:00 GMT+0200. 2014.
- [27] M. open source contributors. *Briar*. GitLab. Available at: <https://code.briarproject.org/briar/briar>. Accessed on: 2025-11-03 21:00 GMT+0200. 2018.
- [28] D. Inyangson, S. Radway, T. M. Jois, N. Fazio, and J. Mickens. “Amigo: Secure Group Mesh Messaging in Realistic Protest Scenarios”. In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. CCS '25. Preprint available at <https://eprint.iacr.org/2024/1872>. Accessed on: 2025-11-03 21:00 GMT+0200. Taipei, Taiwan: Association for Computing Machinery, 2025. DOI: 10.1145/3719027.3765133. URL: <https://doi.org/10.1145/3719027.3765133>.
- [29] R. Friedman, A. Kogan, and Y. Krivolapov. “On Power and Throughput Tradeoffs of WiFi and Bluetooth in Smartphones”. In: *IEEE Transactions on Mobile Computing* 12.7 (2013), pp. 1363–1376. DOI: 10.1109/TMC.2012.117.
- [30] Supermicro. *What is 5G RAN?* Internet. Available at: <https://www.supermicro.com/en/glossary/5g-ran>. Accessed on: 2026-07-15 15:00 GMT+0200. 2026.
- [31] *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications (IEEE Std 802.11-2012)*. IEEE Standard. Archived from the original on 2022-09-14, retrieved 2024-07-30. New York, NY: The Institute of Electrical and Electronics Engineers, Inc, 2012. URL: <https://web.archive.org/web/20220914150307/https://standards.ieee.org/ieee/802.11/4523/>.
- [32] D. Camps-Mur, A. Garcia-Saavedra, and P. Serrano. “Device-to-device communications with Wi-Fi Direct: overview and experimentation”. In: *IEEE Wireless Communications* 20.3 (2013), pp. 96–104. DOI: 10.1109/MWC.2013.6549288.

- [33] B. Strobel. “Opportunistic Networking on Contemporary End User Devices: Augmenting Emergency Notification Systems”. Unpublished master’s thesis. MA thesis. Munich: Technische Universität München, 2024.
- [34] Apple. *How to use AirDrop on iPhone and iPad*. Tutorial. Available at: <https://support.apple.com/en-us/119857>. Accessed on: 2026-17-07 11:00 GMT+0200. 2026.
- [35] S. Saloni and A. Hegde. “WiFi-aware as a connectivity solution for IoT pairing IoT with WiFi aware technology: Enabling new proximity based services”. In: *2016 International Conference on Internet of Things and Applications (IOTA)*. 2016, pp. 137–142. doi: 10.1109/IOTA.2016.7562710.
- [36] Apple. *Highlights of new technologies introduced at WWDC25*. Article. Available at: <https://developer.apple.com/documentation/updates/wwdc2025>. Accessed on: 2026-17-07 11:00 GMT+0200. 2025.
- [37] J. Ott, D. Kutscher, and C. Dwertmann. “Integrating DTN and MANET routing”. In: *Proceedings of the 2006 SIGCOMM Workshop on Challenged Networks*. CHANTS ’06. Pisa, Italy: Association for Computing Machinery, 2006, 221–228. ISBN: 159593572X. doi: 10.1145/1162654.1162659. URL: <https://doi.org/10.1145/1162654.1162659>.
- [38] Corona Warn App. CORONA-WARN-APP SOLUTION ARCHITECTURE. https://github.com/corona-warn-app/cwa-documentation/blob/main/solution_architecture.md. Accessed on: 2026-17-07 11:00 GMT+0200.
- [39] Chee Yi Ong. *Android BLE: The Ultimate Guide to Bluetooth Low Energy*. <https://punchthrough.com/android-ble-guide/>. Accessed on: 2026-17-07 11:00 GMT+0200.
- [40] J. Tosi, F. Taffoni, M. Santacatterina, R. Sannino, and D. Formica. “Performance Evaluation of Bluetooth Low Energy: A Systematic Review”. In: *Sensors* 17.12 (2017). ISSN: 1424-8220. doi: 10.3390/s17122898. URL: <https://www.mdpi.com/1424-8220/17/12/2898>.
- [41] A. Langley, A. Riddoch, A. Wilk, A. Vicente, C. Krasic, D. Zhang, F. Yang, F. Kouranov, I. Swett, J. Iyengar, J. Bailey, J. Dorfman, J. Roskind, J. Kulik, P. Westin, R. Tenneti, R. Shade, R. Hamilton, V. Vasiliev, W.-T. Chang, and Z. Shi. “The QUIC Transport Protocol: Design and Internet-Scale Deployment”. In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. SIGCOMM ’17. Los Angeles, CA, USA: Association for Computing Machinery, 2017, 183–196. ISBN: 9781450346535. doi: 10.1145/3098822.3098842. URL: <https://doi.org/10.1145/3098822.3098842>.
- [42] M. Allman, V. Paxson, and E. Blanton. *RFC 5681: TCP Congestion Control*. USA, 2009.
- [43] J. Lakkakorpi, M. Pitkänen, and J. Ott. “Adaptive routing in mobile opportunistic networks”. In: *Proceedings of the 13th ACM International Conference on Modeling, Analysis, and Simulation of Wireless and Mobile Systems*. MSWIM ’10. Bodrum, Turkey: Association for Computing Machinery, 2010, 101–109. ISBN: 9781450302746. doi: 10.1145/1868521.1868539. URL: <https://doi.org/10.1145/1868521.1868539>.
- [44] I. Keidar, A. Lewis-Pye, E. Shapiro, and N. Talmon. *Constitutional Consensus for Democratic Governance*. 2026. arXiv: 2505.19216 [cs.DC]. URL: <https://arxiv.org/abs/2505.19216>.

- [45] Holepunch. *udx-native: reliable, multiplexed, and congestion-controlled streams over UDP*. Source code repository. Available at: <https://github.com/holepunchto/udx-native>. Accessed on: 2026-07-19 11:00 GMT+0200. 2026.
- [46] T. Perrin. *The Noise Protocol Framework*. Revision 34. Available at: <https://noiseprotocol.org/noise.html>. Accessed on: 2026-07-19 11:00 GMT+0200. 2018.
- [47] A. Zafar, M. A. Hafeez, and A. Munir. *LSEG: A Lightweight and Secure Key Exchange Protocol for Smart Grid Communication*. 2025. arXiv: 2511.07548 [cs.CR]. URL: <https://arxiv.org/abs/2511.07548>.
- [48] C. Boldrini, K. Lee, M. Önen, J. Ott, and E. Pagani. "Opportunistic networks". In: *Computer Communications* 48 (2014). Opportunistic networks, pp. 1–4. ISSN: 0140-3664. DOI: <https://doi.org/10.1016/j.comcom.2014.04.007>. URL: <https://www.sciencedirect.com/science/article/pii/S0140366414001431>.
- [49] N. Alsulami, A. Cherif, and A. Imine. "Collaborative editing over opportunistic networks". In: *Int. J. Ad Hoc Ubiquitous Comput.* 39.3 (Jan. 2022), 141–156. ISSN: 1743-8225. DOI: [10.1504/ijahuc.2022.121121](https://doi.org/10.1504/ijahuc.2022.121121). URL: <https://doi.org/10.1504/ijahuc.2022.121121>.
- [50] R. Pozza, M. Nati, S. Georgoulas, K. Moessner, and A. Gluhak. "Neighbor Discovery for Opportunistic Networking in Internet of Things Scenarios: A Survey". In: *IEEE Access* 3 (2015), pp. 1101–1131. DOI: [10.1109/ACCESS.2015.2457031](https://doi.org/10.1109/ACCESS.2015.2457031).
- [51] V. Del Duca Almeida, A. B. Oliveira, D. F. Macedo, and J. M. S. Nogueira. "Performance evaluation of MANET and DTN routing protocols". In: *2012 IFIP Wireless Days*. 2012, pp. 1–6. DOI: [10.1109/WD.2012.6402866](https://doi.org/10.1109/WD.2012.6402866).
- [52] M. Schmidheiser and M. Radenkovic. *Distributed Accountability in Democracy: Using MANETs and DTNs in the Face of Acts of Questionable Legality*. 2025. arXiv: 2512.23653 [cs.NI]. URL: <https://arxiv.org/abs/2512.23653>.
- [53] P. Juang, H. Oki, Y. Wang, M. Martonosi, L. S. Peh, and D. Rubenstein. "Energy-efficient computing for wildlife tracking: design tradeoffs and early experiences with ZebraNet". In: *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS X. San Jose, California: Association for Computing Machinery, 2002, 96–107. ISBN: 1581135742. DOI: [10.1145/605397.605408](https://doi.org/10.1145/605397.605408). URL: <https://doi.org/10.1145/605397.605408>.
- [54] A. Pentland, R. Fletcher, and A. Hasson. "DakNet: rethinking connectivity in developing nations". In: *Computer* 37.1 (2004), pp. 78–83. DOI: [10.1109/MC.2004.1260729](https://doi.org/10.1109/MC.2004.1260729).
- [55] R. Chao. *FireChat Wasn't Meant For Protests. Here's How It Worked (Or Didn't) at Occupy Central*. Article on MIT Technology Review. Available at: <http://techpresident.com/news/25304/firechat-wasn%E2%80%99t-meant-protests-here%E2%80%99s-how-it-worked-or-didn%E2%80%99t-occupy-central>. Accessed on: 2015-02-22 21:00 GMT+0200. 2014.

- [56] M. R. Albrecht, J. Blasco, R. B. Jensen, and L. Mareková. *Mesh Messaging in Large-scale Protests: Breaking Bridgefy*. Cryptology ePrint Archive, Paper 2021/214. 2021. doi: 10.1007/978-3-030-75539-3_16. URL: <https://eprint.iacr.org/2021/214>.
- [57] S. Project. *Serval Batphone*. GitHub. Available at: <https://github.com/servalproject/batphone>. Accessed on: 2026-21-07 21:00 GMT+0200. 2014.
- [58] F. Legendre. “30 years of ad hoc networking research: what about humanitarian and disaster relief solutions? what are we still missing?” In: *Proceedings of the 1st International Conference on Wireless Technologies for Humanitarian Relief*. ACWR ’11. Amritapuri, Kollam, Kerala, India: Association for Computing Machinery, 2011, p. 217. ISBN: 9781450310116. doi: 10.1145/2185216.2185279. URL: <https://doi.org/10.1145/2185216.2185279>.
- [59] Facebook. *Meta Reports Second Quarter 2026 Results*. Facebook. Available at: https://s21.q4cdn.com/399680738/files/doc_financials/2026/q2/Meta-06-30-2026-Exhibit-99-1-FINAL.pdf. Accessed on: 2026-21-07 21:00 GMT+0200. 2026.
- [60] S. Zuboff. “The age of surveillance capitalism”. In: *Social theory re-wired*. Routledge, 2023, pp. 203–213.
- [61] U. Jeong, L. H. X. Ng, K. M. Carley, and H. Liu. “Navigating decentralized online social networks: An overview of technical and societal challenges in architectural choices”. In: *arXiv preprint arXiv:2504.00071* (2025).
- [62] E. Shapiro. *Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots*. 2026. arXiv: 2511.03286 [cs.DC]. URL: <https://arxiv.org/abs/2511.03286>.
- [63] E. Biagioni. “Early Review of The BitChat Protocol”. In: *2026 International Conference on Computing, Networking and Communications (ICNC)*. 2026, pp. 339–344. doi: 10.1109/ICNC68183.2026.11416837.
- [64] O. Eitan, I. Keidar, and E. Shapiro. *Securing People and their Machines Against Major Faults*. 2026. arXiv: 2607.02304 [cs.DC]. URL: <https://arxiv.org/abs/2607.02304>.
- [65] E. Shapiro. “GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI”. In: *Electronic Proceedings in Theoretical Computer Science* 450 (2026), pp. 119–133. ISSN: 2075-2180. doi: 10.4204/eptcs.450.11. URL: <http://dx.doi.org/10.4204/EPTCS.450.11>.
- [66] M. M. U. Hasan and M. Radenkovic. “Improving Resiliency of Vital Services in Flood-Affected Regions of Bangladesh Using Next-Generation Opportunistic DTN Edge Ad Hoc Networks”. In: *arXiv preprint arXiv:2511.15710* (2025).
- [67] International Telecommunication Union (ITU). *Mobile phone ownership*. Tech. rep. Geneva, Switzerland: International Telecommunication Union, 2025. URL: <https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-mobile-phone-ownership/>.
- [68] o.-s. c. Google. *Flutter*. GitHub. Available at: <https://flutter.dev/>. Accessed on: 2026-08-03 21:00 GMT+0200. 2017.
- [69] o.-s. c. Meta. *React Native*. GitHub. Available at: <https://reactnative.dev/>. Accessed on: 2026-08-03 21:00 GMT+0200. 2015.

- [70] E. Shapiro. “Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp”. In: *3rd International Workshop on Open Challenges in Online Social Networks*. HT ’23. ACM, 2023, 14–21. DOI: 10.1145/3599696.3612898. URL: <http://dx.doi.org/10.1145/3599696.3612898>.
- [71] P. Sanders and K. Mehlhorn and M. Dietzfelbinger and R. Dementiev. “Collective Communication and Computation”. In: *Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox*. Springer Nature, 2019, pp. 393–418.
- [72] Y. Mahéo, F. Guidec, and C. Noûs. “CRDT-based Collaborative Editing in OppNets: a Practical Experiment”. In: *17th Conference on Mobile Ubiquitous Computing, Systems, Services and Technologies (UBICOMM 2023)*. Ed. by J. L. Mauri. Porto, Portugal: IARIA, Sept. 2023, pp. 13–21. URL: <https://hal.science/hal-04249567>.
- [73] P. T. Tran-Truong, V. X. Q. Nguyen, H. X. Son, P. Nguyen-Ngoc, K. H. Vo, and T. M. Nguyen. *Systematic Survey on Privacy-Preserving Architectures for IoT and Vehicular Data Sharing: Techniques, Challenges, and Future Directions*. 2026. arXiv: 2603.01876 [cs.CR]. URL: <https://arxiv.org/abs/2603.01876>.
- [74] E. Shapiro. “Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI”. In: *arXiv preprint arXiv:2602.06934* (2026).
- [75] E. Shapiro. “GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language”. In: *arXiv preprint arXiv:2510.15747* (2025).
- [76] E. Shapiro. “Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations”. In: *Proceedings of the 27th International Conference on Distributed Computing and Networking*. arXiv preprint arXiv:2502.11299. 2026, pp. 71–81. DOI: 10.1145/3772290.3772309.
- [77] A. Keränen, C. Holmberg, and J. Rosenberg. *Interactive Connectivity Establishment (ICE): A Protocol for Network Address Translator (NAT) Traversal*. RFC 8445. 2018. URL: <https://www.rfc-editor.org/rfc/rfc8445>.